CTVS-1

Synchronous Tokenized Vaults on Cardano

Authenticated identity, integer share accounting, and an exact candidate ledger boundary.

Technical whitepaper / 29 September 2026

Prepared for Steer Protocol and ecosystem review.

Companion: CTVS-2.

Publication and evidence status

The reference encoding is CTVS-WIRE-2, wire integer 2, Direct Custody Profile (wire profile ID 0).

A proposed standard, not an adopted CIP or frozen deployed interface.

BASELINE REPOSITORY COMMIT

`f5f782de9ba6a7bf6cbb8213995a31734888559c`

Abstract

Tokenized vaults represent pooled economic exposure through fungible shares. A useful standard lets wallets, allocators, lending applications, and analytics providers identify the underlying asset, determine what the shares represent, construct bounded transactions, and reconstruct the outcome without understanding each vault's internal strategy.

CTVS-1 proposes this boundary for Cardano. It specifies authenticated vault identity, economic share supply, backing and liability accounting, conversion rules, direct deposit and redemption operations, and reproducible infrastructure views. It translates interface guarantees into predicates over extended-unspent-transaction-output transactions rather than reproducing an account-based contract interface. A separate asynchronous standard, CTVS-2, reuses the accounting model while separating request creation, economic settlement, and delivery.

The reference profile uses immutable configuration, one authenticated accounting-state UTxO, direct custody, native mint/burn shares, fixed entry and exit fees, and explicit ADA storage reserves. Its arithmetic uses integer rounding and immutable virtual balances. A restricted transaction-composition profile defines the candidate obligation domain; unrestricted cross-vault composition is not assumed. Managed strategies, authenticated share inventory, and governed terms are extension profiles with additional proof obligations.

The central requirement is economic conservation under authenticated transitions. An implementation must justify every change in backing, supply, liabilities, and user ownership. Conformance must also expose permissioning, valuation trust, liquidity limits, and recovery assumptions. A common interface does not make a strategy solvent, a price report accurate, or an operator available.

This paper specifies exact object, role and recursive Data rules for the Direct Custody Profile (wire profile ID `0`). It separately specifies native-format execution conformance and a minimum common integration contract. Economic supply and verifiable ownership are common requirements; the ID/STATE token construction, single-state custody, virtual-balance formula and restricted-purpose transaction family are explicit reference choices. Native formats are not accepted on the strength of matching JSON alone.

Common guarantees and explicit implementations

CTVS distinguishes **common economic and integration requirements** from the **fully specified reference construction**, defines a bounded route for native-format conformance, and specifies a semantic direct-at-settlement delivery variant. It does not add a universal transcript language, module framework, permission language, routing engine, or hosted adapter runtime.

Component	Specification
Reference wire	CTVS-WIRE-2; magic CTVS, integer 2, Direct Custody Profile (profile ID `0`)
Reference schemas and fixtures	Exact encodings and fixtures in `candidate-wire/`
Native integration	Explicit semantic mapping and evidence required; no native implementation certified here
Direct-at-settlement delivery	Specified semantic variant outside the Direct Custody Profile; no new wire tag or assigned profile integer
Budget fields	Existing storage, budget and exact-fee fields retained; no silent simplification

Profile boundaries. The Direct Custody Profile fixes its Request spending rules, constructors, and terms-commitment bytes. Native mappings and `delivery_at_settlement` have no assigned on-chain profile here. A native implementation or a distinct reference-wire variant must bind its own reviewed encoding and execution rules. A capability label cannot reinterpret an existing datum field or action.

A conformance claim must identify the actual implementation, supported operation, and evidence. Publication alone certifies none of them.

Implementation status. The historical economic and codec results are retained with their original limits in historical validation evidence. The linked Direct Custody Profile reference has compiled Aiken checks, signed local transaction scenarios, and an accepted-chain reader with local tests [19], [21]. Those are implementation results, not proof of node-backed acceptance, deployment, independent interoperability, or a completed external audit. The illustrative response schema from the earlier source bundle is not claimed to match the current reader byte for byte.

CTVS permits native contracts and datums where they satisfy the common guarantees. Its conformance and delivery requirements are proposals, not claims of verified SundaeSwap conformance.

Figure index and reading key

The figures illustrate the economic and wire rules. The integration response contract is stated in this paper. The reference implementation is linked at a fixed repository commit; its local validation is not a deployment, independent audit, or adoption of CTVS as a CIP.

The figures use a shared spacing grid, thin uniform outlines, restrained line icons and soft blue, violet, green and sand cards. Labels and captions identify the applicable profile. Colour supports grouping; it does not establish authority, success, or a new protocol rule. Arrows identify the relationship named in the figure, not the provenance of individual fungible coins. Dashed treatment marks the locally stated alternative or unsupported path.

Eleven figures are vector SVG/PDF illustrations with selectable text; four figures are image-based. The source package provides the artwork used in the build.

Figure	Placement	Primary question
1. One standard boundary, two implementation paths	Section 2.5	How can the Direct Custody Profile and a native implementation satisfy the same guarantees?
2. Authenticated objects and the economic boundary	Section 4.1	What are Config, State, Request, and Claim?
3. Acyclic deployment and identity creation	Section 4.2	How are the trust-root objects created without a hash cycle?
4. Protected obligations and output allocation	Section 8.1	Why can one output not satisfy two obligations?
5. Direct deposit and redeem anatomy	Sections 9.2-9.3	How do the direct operations affect backing, fees, supply, and payouts?
6. ADA accounting partitions	Section 9.4	Which lovelace is backing, fee liability, reserve, or external funding?
7. Managed accounting boundary	Section 11.1	What extra evidence is required once backing leaves direct custody?

The diagrams explain the written requirements and add no validation evidence. The linked reference implementation has its own dated local results [21]; node-backed acceptance, managed integration, independent review, and deployment remain open.

Contents

Abstract 2

Common guarantees and explicit implementations 3

Figure index and reading key 4

1. Motivation and scope 6

2. Standard architecture and conformance profiles 7

3. Ledger and transaction model 11

4. Identity, deployment, and state succession 12

5. Accounting definitions and boundaries 16

6. Conversion, fees, and limits 18

7. Direct transitions and authorization 24

8. Payment attribution and reference composition 26

9. Worked UTxO transactions 28

10. Datum, redeemer, and encoding specification 32

11. Managed accounting and additional fee models 37

12. Formal properties and proof sketches 40

13. Common integration contract and native interpretation 42

14. Resource limits, operational behavior, and risks 44

15. Validation and release requirements 45

16. References 47

Appendix A. Scope and delivery checklist 49

Appendix B. Candidate Price encoding 50

1. Motivation and scope

1.1 The interoperability objective

The useful outcome of a tokenized-vault standard is a reduction in integration-specific interpretation. An application should not need a new accounting model for every vault before it can ask what asset is accepted, how shares are priced, whether an operation is available, what it will cost, and which transaction effects must be verified.

ERC-4626 standardizes a single-asset vault interface, share conversions, previews, limits, and four economic operations. ERC-7540 adds request-based entry and exit. These are semantic precedents; CTVS does not claim ABI compatibility or exact equivalence with every EVM field. In particular, this paper deliberately names share backing separately from gross managed assets. [2], [3]

On Cardano, the common boundary must additionally identify the correct UTxOs, their authentication rules, datum and redeemer schemas, required witnesses, output obligations, and chain point used by an off-chain quote. A JSON endpoint alone does not provide this boundary.

1.2 In scope and outside the standard

Scope rule. CTVS standardizes authenticated economic meaning and enforceable user guarantees. It does not require one native order datum, strategy engine, permission mechanism or custody arrangement. Sections explicitly labeled Direct Custody Profile remain exact requirements of that profile. An alternative implementation must satisfy the common requirements through the mapping and evidence in Section 2.5; it does not inherit the reference profile's proofs by analogy.

The core boundary has one underlying asset and one transferable native share asset. Deposits and redemptions are denominated in the underlying asset. Internal exposure may be more complex, but an extension must justify its valuation and liquidity semantics.

Standardized boundary	Implementation-specific interior
Vault and asset identity; supported profiles	Strategy selection and portfolio construction
Economic backing, supply, fees, and limits	Lending, liquidity positions, hedges, and leverage
Bounded user operations and ownership changes	Protocol-specific position data and execution routes
Authenticated accounting and valuation provenance	Choice of oracle, manager, or authorized operator
Discoverable schemas and reconstructible lifecycle	Commercial service providers and governance organization

A multi-asset entry router may swap into the underlying asset before depositing, but the swap is not a CTVS-1 deposit. A share's suitability as collateral is a separate risk decision. Permissioned deposits or strategy execution do not imply restrictions on ordinary native-share transfers; programmable transfer restrictions require a different, explicitly supported asset model.

1.3 Normative language and interpretation

Normative scope is part of each requirement. Common requirements bind every claimed CTVS behavior. Reference-construction requirements bind only CTVS-WIRE-2 Direct Custody Profile. An extension or native mapping binds only the behaviors it explicitly specifies and proves. A capability label cannot waive a common guarantee; unsupported directions and differing user rights must be reported instead of normalized away.

MUST and MUST NOT express proposed conformance requirements. SHOULD expresses a recommendation with a documented exception. MAY expresses an option only when its profile is advertised. These words do not claim that the repository currently enforces the requirement.

The economic core, reference implementation choices, and experimental extensions are distinct. An implementation MUST list its exact profile combination. Recognition of an extension name is not proof that every combination is valid.

2. Standard architecture and conformance profiles

2.1 Common layers

CTVS has a shared semantic core used by both papers. CTVS-1 adds direct transaction execution; CTVS-2 adds request-based execution. Identity, accounting definitions, and infrastructure interpretation are common supporting specifications rather than an unrelated standard for each application.

Layer	Required responsibility
Identity and descriptor	Bind a deployment to assets, authenticated state, scripts, schemas, and economic terms.
Economic kernel	Define backing, participating supply, rounding, fees, and operation deltas.
Execution profile	Define inputs, witnesses, state updates, payments, and failure conditions.
Accounting profile	Establish which property backs shares and how it is valued.
Provider interface	Return verifiable snapshots, quotes, limits, transaction effects, and lifecycle data.
Conformance evidence	Connect requirements to vectors, validator tests, independent clients, and measured resource bounds.

2.2 Exact reference profile - not the universal architecture

The CTVS-WIRE-2 Direct Custody Profile combination, descriptively `DC-MB-IMM-RCI`, means direct custody, native mint/burn supply, immutable terms, and restricted composition inputs. It uses an enterprise state address, so there is no staking credential attached to that state. It does not assume or promise strategy yield. These are reference choices, not universal requirements for every future CTVS vault.

Axis	Reference decision	Extension boundary
Custody and valuation	All backing is in authenticated state	Verified positions or attested NAV
Supply	Native mint/burn; supply counter equals ledger supply	Authenticated unissued inventory
Terms	Immutable economic configuration	Governed terms and protected migration
Fees	Fixed entry/exit asset fees	Management/performance fee profiles
Operation	Exact-asset deposit/withdraw; exact-share mint/redeem	CTVS-2 request execution
Composition	One vault accounting transition; closed script-input family	Reviewed direct-script and cross-vault allocation adapters

Shutdown	No terminal close or reserve sweep	Separately specified close/migration proof

SundaeSwap's published architecture supports script/multisig-owned orders and authorized scoopers. Minswap V2 specifies authorized batchers and pre-minted LP inventory. These examples motivate extensibility; they do not certify CTVS compatibility or current deployed bytecode. [9], [10]

2.3 Capability disclosure

Deposit and redemption capabilities MUST be independent. A vault may expose direct deposits and asynchronous redemptions, or the reverse. All four CTVS-1 operations are required for a fully direct reference deployment; an asynchronous-only direction MUST NOT pretend to offer an executable direct operation.

Read conformance, direct-execution conformance, request-execution conformance, and optional accounting extensions are separate claims. A provider may display a vault it cannot safely execute against, provided that distinction is explicit.

2.4 Reference wire and compatibility boundaries

CTVS-WIRE-2 remains the only exact **reference** wire in this publication. Wire integer 2, profile integer 0, a native implementation version, deployed script hashes and evidence records are separate identities.

The Direct Custody Profile still fixes direct custody, mint/burn supply, immutable terms, the Section 6 reference fee/math rules, key roles, restricted composition and full-fill/snapshot/funded-claim asynchrony. Modes remain direct entry 1, direct exit 2, async entry 4 and async exit 8. `execution_modes` is a nonzero mask in 1..15; direct entry includes deposit and mint, direct exit includes withdraw and redeem. `max_batch` remains 1..16 as a provisional structural cap, not measured capacity. No direct-at-settlement, script-controller or managed-update action has been inserted into this encoding.

Native execution conformance below is not a second universal wire. It is a requirement for a specific implementation/version to expose and prove the common semantics using its own authenticated representation. Any future modification of the Direct Custody Profile's exact records requires an intentional new wire/profile decision and migration analysis. [12]

Sections 6.5 and 6.6 and Appendix B additionally specify a candidate `Price` interface for verified native conversions and previews. Its encoded version 1 is independent of CTVS-WIRE-2. It does not modify the Direct Custody Profile, certify a native integration, or require every conforming vault to invoke a pricing script.

2.5 Native-format conformance without an adapter platform

A native implementation MAY retain its existing scripts, datums and token identities. It MUST publish a versioned mapping with the following seven elements before claiming execution conformance. A read-only integration that cannot establish these elements remains read-only.

Mapping element	Required result
Implementation identity	Pin native protocol/build, scripts or policy identities, configuration authority and applicable terms. Explain how these are authenticated, including upgrades if any.
Decoding and sources	Identify exact native objects, schemas, state/claim identity and their chain-point-consistent sources. Preserve original ledger identities.

Mapping element	Required result
Economic interpretation	Derive one underlying asset, the share asset, participating supply, net backing, liabilities and available liquidity without double counting.
Operations	Specify exact input/output semantics, executable formulas, rounding, fees, limits, pricing basis and ownership cutover for each supported action.
Authorization and allocation	Identify the executing guards and required witnesses; prove every user-bound destination and non-overlapping value obligation.
Construction and recovery	Supply reproducible native transaction recipes, prerequisites, failure statuses, cancellation and delivery rights. A proprietary builder endpoint alone is insufficient.
Evidence	Bind positive/negative byte and transaction fixtures, independent interpretation and the actual conformance scope to the pinned implementation.

For full direct conformance, the four operations and their applicable views are available; a partial integration must list its exact supported operations and cannot claim the missing ones. Async-only directions retain their shared economics but do not fake a synchronous operation. Ordinary transferable shares are required by this boundary; an account balance without such a share is not a CTVS tokenized vault merely because a provider displays it similarly.

Mappings translate representation, not rights. A mapping cannot make an unfunded ticket a funded claim, invent fixed refund rights, hide a different fee convention, or call a multi-asset withdrawal a guaranteed single-underlying redemption. If a wrapper actually creates the missing guarantees, identify that wrapper as a distinct implementation and disclose its custody and trust boundary.

This is a finite conformance record and reviewed construction procedure, not a universal translator or code-execution system. The descriptor does not authorize downloaded programs. No generic module catalogue, transcript interpreter, cross-protocol routing engine or new permission language is required.

A mapping using `Price` MUST publish its source quantities and denominations, any fixed-point rate scale, exact intermediate rounding, supported assertions, evidence authentication and approved provider identity. Scale changes require an authenticated configuration/version transition, not a change to display metadata. The off-chain response provider in Section 13 and the on-chain pricing validator in Section 6.6 have distinct responsibilities; neither establishes the other’s conformance.

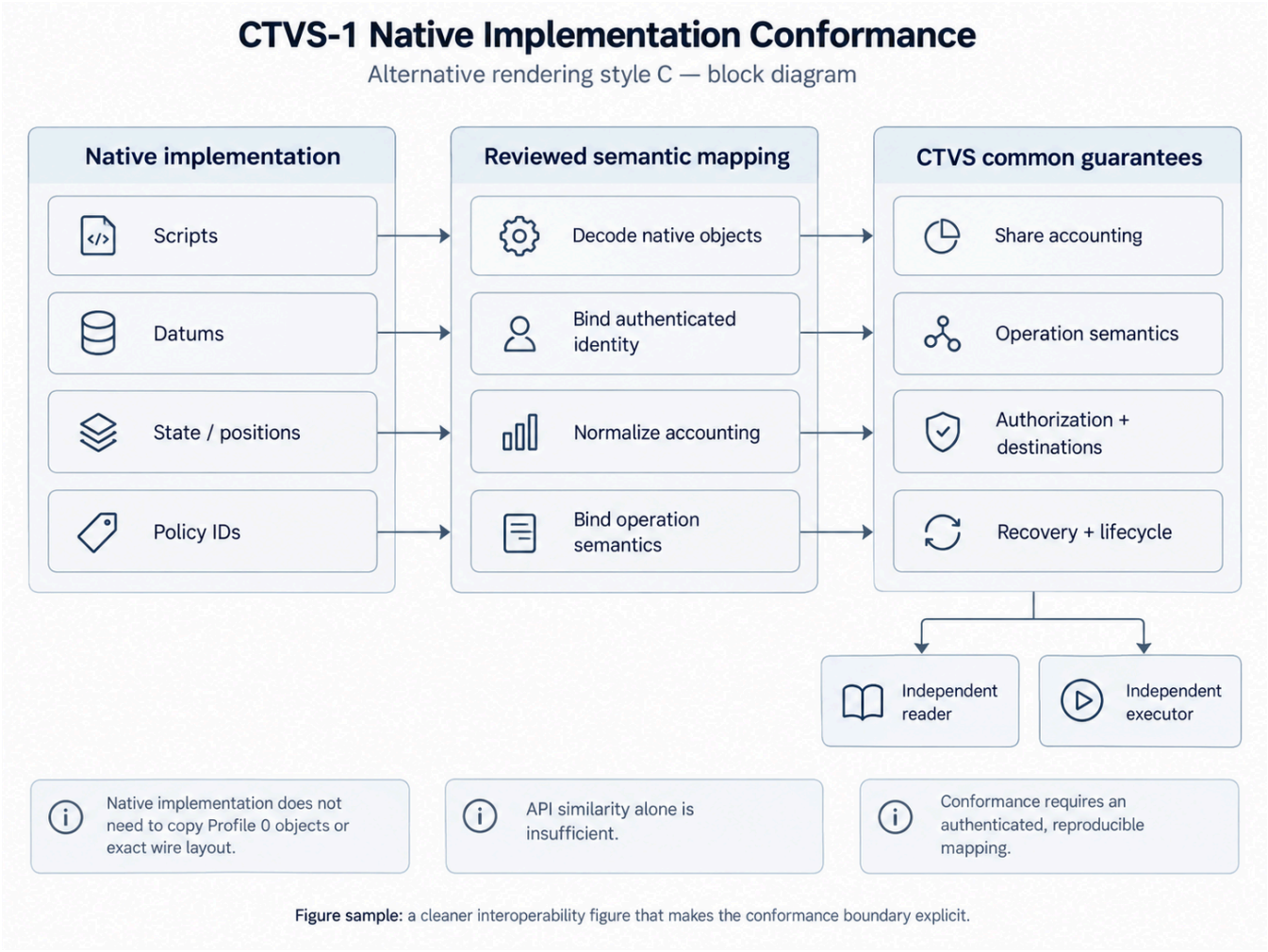


Figure 1: *One standard boundary, two implementation paths. A native protocol may retain its own scripts and datums, but execution conformance requires a reviewed mapping to the same authenticated economic guarantees. Matching JSON or labels alone is not sufficient.*

2.6 Common guarantees and implementation-local mechanisms

Common guarantee	Concrete Direct Custody Profile choice
Authentic vault, terms and accounting evidence	One policy with ID/STATE/SHARE and the specified Config/State objects
Conservation of participating shares and backing	Native mint/burn and all backing in one state UTxO
Reproducible conversions, fees and user bounds	Fixed virtual balances and entry/exit markup equations
Actual, scoped authorization and complete payment attribution	Key roles, disjoint outputs and restricted input/purpose family
Explicit request ownership, pricing and recovery	Full-fill snapshot requests; independent funded claims
No loss of outstanding rights during updates or closure	Immutable terms, permanent identities and no terminal sweep

The right column remains normative for the Direct Custody Profile and must not be weakened to accommodate another system. The left column defines what a separately specified native implementation must demonstrate. Single-

state concurrency, exact token names, a particular authorization purpose and absence of a closure branch are not universal requirements by themselves.

3. Ledger and transaction model

3.1 Objects and notation

Let the ledger UTxO set be a partial map from output references to outputs. An output reference is a transaction hash and output index. An output contains an address, a multi-asset value, a datum representation, and optionally a reference script. For this paper, every protocol state or request datum is inline.

$$\mathcal{L} : \text{OutRef} \rightharpoonup \text{Output}; \qquad u = (\text{address}, \text{value}, \text{datum}, \text{scriptRef}) \tag{1}$$

An asset identifier is either ADA or a pair of policy ID and asset name. Write $v(\alpha)$ for the integer quantity of asset α in value v. An action transaction contains ordinary inputs I, reference inputs J, outputs O, a signed mint vector M, witnesses, redeemers, fees, and a validity interval.

CIP-31 reference inputs make existing outputs readable without spending them. CIP-32 specifies inline datums; CIP-33 supports reference scripts. A reference is evidence available to a validator, not authority to change the referenced object. [3–5]

3.2 Conservation

For successful transactions in the restricted profile, which excludes withdrawals, certificates, governance actions, treasury-related fields, and other additional balance sources or sinks, the ledger value equation is:

$$\sum_{i \in I} \text{value}(i) + M = \sum_{o \in O} \text{value}(o) + \text{fee } \mathbf{1}_{\text{ADA}}. \tag{2}$$

This ledger equation is necessary but insufficient. A transaction can balance while paying the wrong recipient. CTVS validators must additionally establish ownership, allocation, accounting, and supply invariants.

Ordinary key inputs require their ledger authorization. Script inputs require successful validation of their spending conditions. Merely including a script as a reference input does not execute an authorization proof. Invalid-script collateral is separate from successful economic accounting; a failed transaction can affect collateral under the ledger's collateral rules. [8]

3.3 Atomicity and concurrency

A successful direct action consumes a particular state UTxO and creates a successor. A competing transaction using the consumed reference must be rebuilt. This is state-version contention, not a statement that Cardano universally allows only one transaction per block.

A quote is therefore attached to an output reference, not just a wall-clock timestamp. Reference inputs reduce read contention; they do not permit two independent spends of one state. Independent requests in CTVS-2 move user submission outside this contested update.

3.4 Responsibility and observable evidence

Requirement	Responsible boundary
Existence, no double-spend, successful value balance, valid witnesses, collateral, and era rules	Cardano ledger; represented faithfully in test contexts
Own-input resolution, protected object identity, value/datum interpretation, and branch obligations	Spending validator and minting policy
Privileged role	Validator checks an explicitly required signatory; ledger validates its witness
Actual network, complete serialized size, fees, minimum ADA, collateral construction	Builder and ledger, not a self-reported datum
Discovery, origin attribution, authentic claim lineage, coherent snapshots, rollback	Indexer/integrator, without replacing on-chain authorization

The pinned Aiken transaction view is not the complete serialized transaction. Its fields include certificates, withdrawals, votes, proposals, current-treasury declarations, and donations. Omitting a field from a symbolic model does not prove it harmless. The Direct Custody Profile requires empty withdrawals, certificates, votes, and proposals, and absent current-treasury and donation fields. A zero-withdrawal aggregation hook is excluded too. Non-genesis minting is restricted to the configured SHARE delta. Unsupported purposes fail. [14], [18]

A funding witness and the script-visible required-signatory list are not interchangeable. Builders MUST explicitly request each privileged signature checked by a validator. A request's controller is not inferred from funding inputs, a receiver, metadata, or an EVM-like caller.

The semantic transaction model uses symbolic ledger/signatory information. The wire-format checks exercise selected shape, role, and linkage predicates. Neither replaces phase-1 ledger checks, complete phase-2 execution, or cryptography. [22], [12]

4. Identity, deployment, and state succession

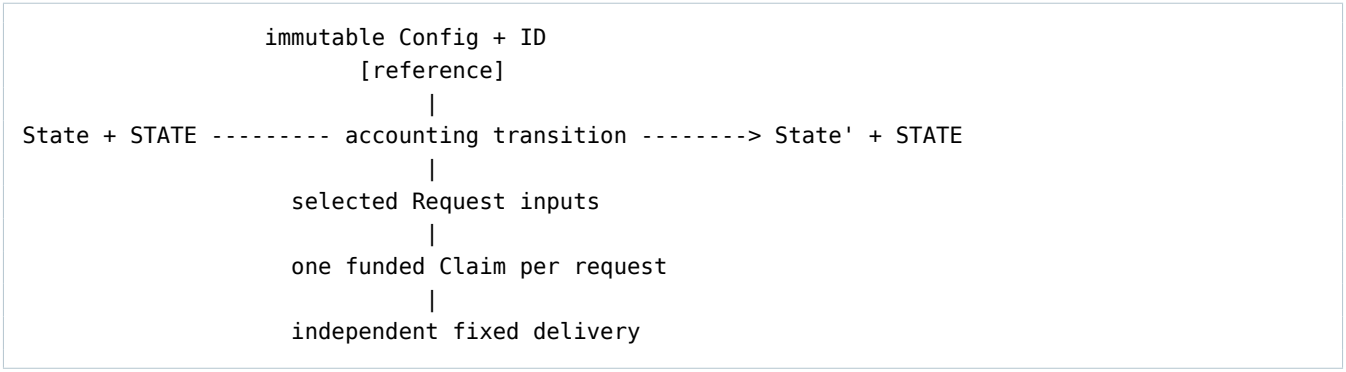
Reference construction. Section 4 specifies Direct Custody Profile's exact authentication and deployment mechanism, not a factory every incumbent must redeploy. Every implementation must establish authentic assets, terms, accounting authority, succession and user-property separation. Section 2.5 requires a native proof of equivalent obligations; it does not accept look-alike metadata in place of the construction below.

4.1 Four objects, four distinct proofs

Object	Authentication and contents	Spending role
Config	One exact ID asset; known immutable lock; committed Terms; declared storage ADA	Read by reference; no ordinary spend
State	One exact STATE asset; expected full vault address; State datum; closed economic value	One authenticated successor for every accounting transition
Request	Originating output reference; supported recovery envelope; valid economics/value when selected	Settlement acknowledgment, fixed cancellation, or fixed expiry refund

Object	Authentication and contents	Spending role
Claim	Exact fixed-delivery script; funded asset/value; valid settlement lineage for authentic indexing	Deliver the entire locked value to the fixed destination

Requests and claims do not need a per-user NFT in this reference profile. This does **not** mean their datums prove creation authority. Anyone can create a script-address output; the output's spending validator is not an admission hook. Providers distinguish candidates from authentic settlements. Reference inputs do not add spendable value and do not execute the referenced output's authorization conditions. [4], [5], [14]



The mutable datum MUST NOT contain supposedly authoritative `pending_deposits`, `pending_request_count`, or a complete queue total unless request creation also updates an authenticated registration mechanism. Independent request creation does not touch state and cannot keep such a counter exact. Pending totals are chain-point-bound indexer views in this profile. The permanent-identity/no-close policy avoids pretending to prove global absence of outstanding requests from one local datum.

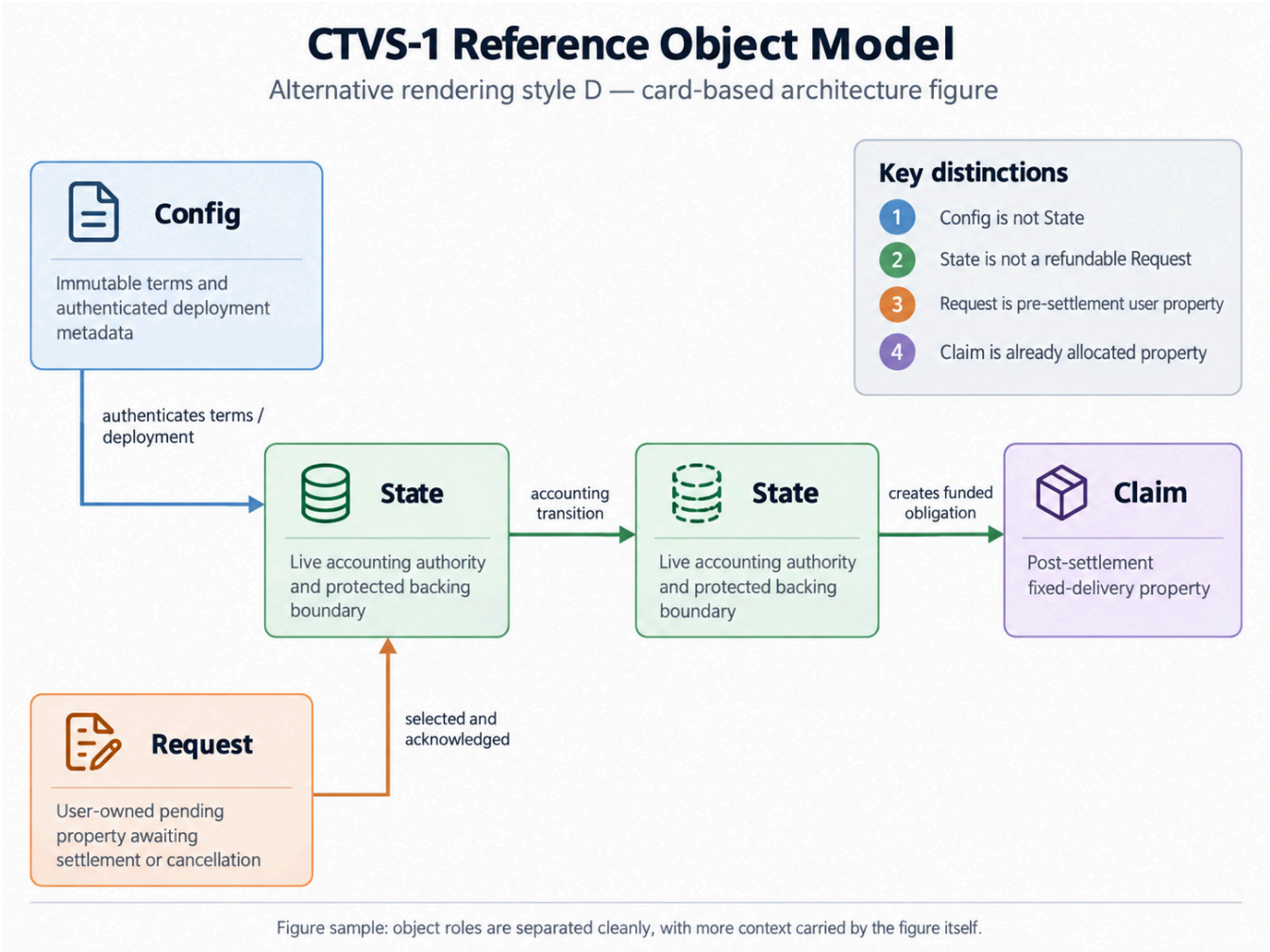


Figure 2: *Authenticated reference objects and the economic boundary. Config, State, Request, and Claim have distinct authentication, ownership, and spending roles. A request-shaped datum cannot route genuine State custody through a refund branch.*

4.2 Acyclic deployment and identity creation

Select and publish three concrete script templates: an always-failing configuration lock F_lock, a fixed-delivery claim script Q_claim, and a per-vault multipurpose program T. F_lock and Q_claim are compiled first. Their identities are fixed in the reviewed T build. They are not arbitrary URLs or caller-selected replacement scripts.

Let r0 be an existing seed output reference. Let K be Terms, excluding any identity that depends on T's final per-vault hash. Define:

$$h_K = \text{Blake2b}_{256}(\text{domain} \parallel \text{serialiseData}(K)). \tag{3}$$

Here `domain` is the fixed 14-byte constant `435456532f48322f5445524d5300` (hexadecimal, including its final zero byte). Decode the hexadecimal to bytes before hashing; do not hash the printed characters. The constant is not derived from the public wire-format name. The candidate deployment then applies r0 and hK to T and obtains policy/script hash P. **P is derived using the ledger's versioned script-hash construction, not an ad hoc hash of JSON, source text, or a displayed script hash.** The exact compilation/parameter-application path must still be demonstrated.

The byte names under P are fixed as `ID`, `STATE`, and `SHARE`. This reference construction uses P both for the state/request payment script and its minting purpose. The genesis mint branch consumes r0, mints exactly ID(1) and STATE(1), mints no shares, and creates:

- Config at enterprise script F_lock, with seed r0, policy P, Terms K, matching hK, and its exact reserve.
- State at enterprise script P, with sequence zero, A=S=F_fees=0, a positive funded reserve, matching hK, and initial pause flags zero.

It also rejects an underlying asset under P, additional assets minted by initialization, malformed terms, unexpected identities, wrong addresses, attached reference scripts, and nonzero initial economic issuance. The shared claim script Q_claim has no dependency on P and cannot introduce a hash cycle. A descriptor/blueprint must not introduce a cycle by committing to an artifact that embeds P inside K.

The future mint branch permits only SHARE deltas coupled to an authentic state spend. ID and STATE can neither mint again nor burn. Genesis is a one-shot *branch*, not a statement that every future invocation of the share policy is one-shot.

Unclosed proof obligation: the package's repeated-byte fixture hashes are placeholders, not hashes of these compiled scripts. A token under an arbitrary policy is not authenticated merely because its name is STATE. Compiled genesis and script-template verification remain mandatory.

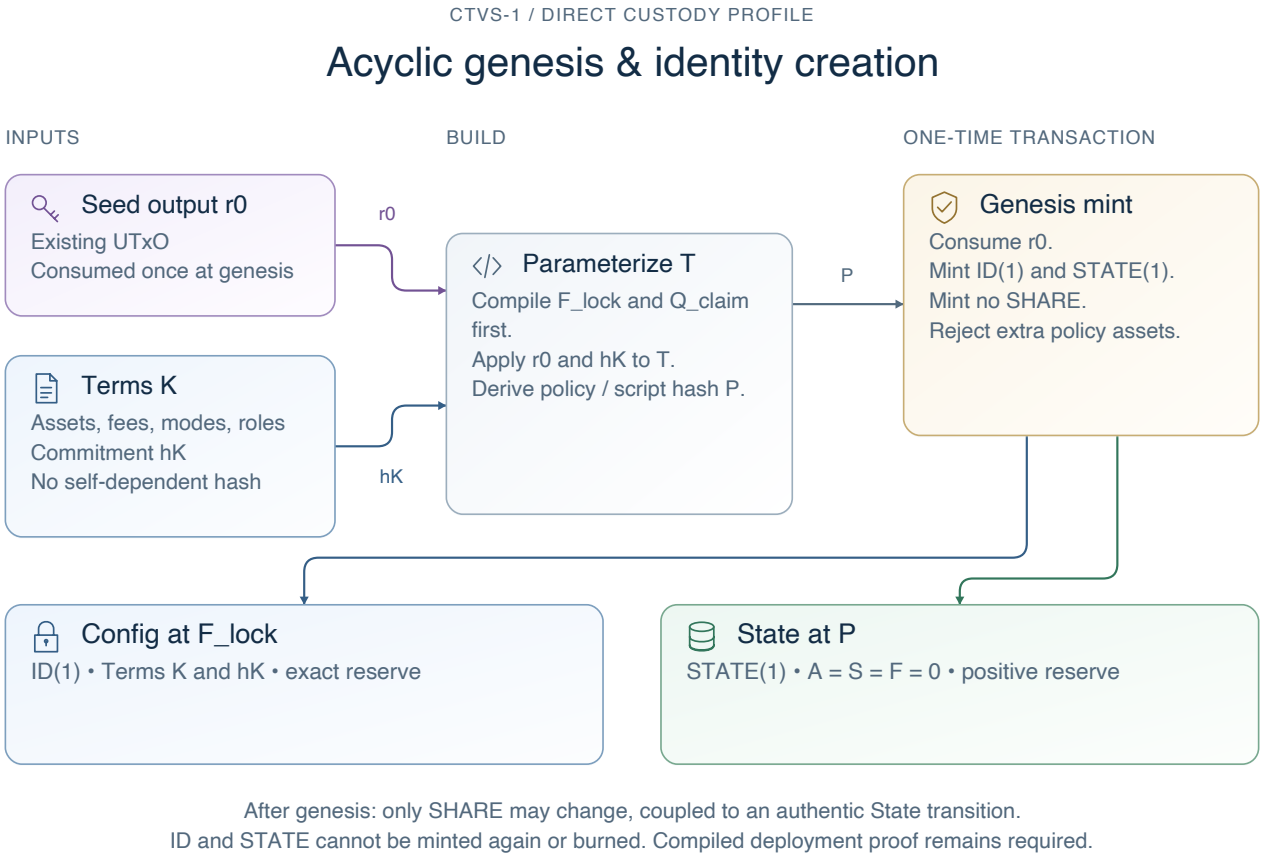


Figure 3: *Acyclic deployment and one-time identity creation. The seed and Terms commitment parameterize the per-vault program; genesis creates Config and zero-supply State. Compiled hash construction and negative genesis vectors remain required evidence.*

4.3 Authenticate the role before datum dispatch

For an ordinary input at the expected vault address, resolve it from the spending purpose's own output reference. First inspect protected identity assets. An input holding STATE(1) must take the State branch and have a State datum. A request-looking datum must never route actual STATE-bearing custody through a generic refund path. Any invalid nonzero STATE quantity or unexpected ID-bearing input fails.

For an input without these identity assets, require the supported request envelope before processing request branches. Do not choose a trusted role only from a user-controlled constructor tag. The tests exercise state-as-request rejection, state look-alikes without the token, wrong full address, and attached reference scripts.

4.4 Continuing state and terminal policy

Resolve the current input through its actual `Spend(own_ref)` purpose. Exactly one ordinary input contains this vault's STATE(1), and exactly one successor retains it at the same required enterprise script address. Authenticate configuration by its ID asset, immutable lock, terms, seed and script parameters; matching two untrusted datum fields is not authentication. The successor must preserve magic, wire version, policy, terms hash, and every field not permitted to change by the selected action. Sequence increases exactly once and must remain in range.

Config/state outputs MUST contain inline data and no attached reference script. The reference script witness, when used, is hosted separately; its location is an availability hint, never an alternative script identity. Normal economic actions preserve state storage. Only the positive reserve-top-up branch increases it. State/ID minting, burns, sweeps, in-place upgrades, and terminal closure are forbidden after genesis.

The permanent config reserve and absence of a sweep are disclosed design costs. Zero supply does not prove global absence of independent requests or claims. Residual backing remains under the same immutable arithmetic and can participate in later entry; no administrator acquires it implicitly. Lost shares remain issued supply. The model's fixture identities do not prove that secure genesis can actually create this construction; compiled trust-root evidence remains outstanding. [12], [22]

5. Accounting definitions and boundaries

5.1 Economic quantities

All economic arithmetic uses base-unit integers. Decimal display settings are presentation metadata, never inputs to the conversion kernel.

An asset quantity is denominated in that asset's ledger units; a share quantity is denominated in the identified share asset's ledger units. These units need not have the same display decimals. A source rate's fixed-point scale is a separate interpretation rule authenticated by its native mapping, not inferred from ticker symbols or mutable display metadata. Section 6.5 specifies conversion between such representations.

Symbol	Definition
A	Underlying-asset units backing economically issued shares
S	Participating economic share supply, including valid user escrows and share claims
F	Accrued asset-denominated fees owed to the fee recipient; excluded from A
R	Non-economic ADA storage reserve
P	Pending deposits not economically admitted

Symbol	Definition
C	Fixed, funded redemption claims no longer backing remaining shares
L	Asset liquidity available for the advertised immediate operation
N	Ledger supply of the native share asset
I_s	Authenticated, economically unissued share inventory, when supported

Pending balances and funded claims are separate UTxOs in the reference request profile. They are not subtracted again from a state balance that never included them.

5.2 Direct-custody identities

For an ADA underlying, the state carries exactly the state token and the following lovelace balance. For a native-token underlying, it additionally carries exactly that underlying asset and no unrelated native assets:

$$V_{\text{state}}(\text{ADA}) = A + F + R, \quad \text{ADA underlying} \tag{4}$$

$$V_{\text{state}}(\alpha) = A + F, \qquad V_{\text{state}}(\text{ADA}) = R, \quad \text{native-token underlying.} \tag{5}$$

The reference value is closed: unaccounted excess assets, reserve reductions, and extra state tokens are invalid. Storage top-ups increase R exactly and have no effect on A, F, or S.

ADA's fungibility means validators prove balance and allocation equations, not the provenance of individual lovelace. Any externally funded output top-up must leave all protected balances and outputs intact. Network fees and collateral must not silently consume backing, accrued fees, or refundable user storage.

5.3 Economic supply first; token mechanics second

Participating economic supply is the common quantity. For every supported issuance mechanism:

$$S' = S + q_{\text{issued}} - q_{\text{extinguished}}. \tag{6}$$

The reference mint/burn mechanism specializes this to $S = N$ and $M(\text{share}) = q_{\text{issued}} - q_{\text{extinguished}}$. No alternative burn path may desynchronize its state counter. A native inventory mechanism instead establishes $S = N - I_s$, with authenticated, economically unissued inventory inaccessible to ordinary holders until valid issuance.

A hypothetical ledger supply of 1,000 shares with 400 verified unissued inventory shares represents 600 participating shares. A native field called `total_lp`, an explorer mint total or a displayed circulation figure MUST NOT be selected as the denominator by its name alone. The mapping must derive the relationship across initialization, issuance, exits and fees.

Transfers do not change economic supply. Under the supported request semantics, pending-redemption shares and shares already issued into claims participate; neither is unissued inventory. Moving a user's shares into escrow cannot relabel them as inventory.

Nonzero gross issuance and redemption may net to zero native mint. The accounting transition must still enforce both gross obligations and all user allocations. Inventory and mint/burn implementations use their own proved supply

reconciliation, not a presumption that the minting policy always executes. Inventory conformance requires its own concrete native trace; it remains outside the Direct Custody Profile.

5.4 State fields versus derived views

The reference State contains only identity/version, terms binding, sequence, backing, economic supply, fees, storage, and pause flags. It contains no free-standing NAV epoch, queue size, pending-asset total, or liquidity oracle. For direct custody, the authoritative pricing checkpoint is the **consumed state reference**, with sequence as supporting lineage. Managed checkpoints need a separately encoded state and accepted-report mechanism.

Pending totals are indexer aggregates at a named chain point because request creation does not mutate this state. They must not be mistaken for validator-enforced capacity reservations. Aggregate value bounds are required in addition to bounds on individual fields: for example, bounded A, F, and R do not alone prove that A+F+R fits the permitted physical-value domain. [12]

6. Conversion, fees, and limits

Common operation contract; explicit reference formulas. Every claimed implementation must publish executable conversion and action rules in underlying/share base units. Fee-free conversions are caller-independent and round down; previews include applicable operation fees, distinguish limits from mathematical output, and identify their state basis. Exact-input actions round economic outputs down; exact-output actions round required economic inputs up under the declared rule. User minima/maxima remain on-chain obligations.

Sections 6.1–6.4 give the complete Direct Custody Profile algorithm. Its virtual asset constant of one, fixed virtual-share parameter and percentage-markup fee are not mandatory economics for all native integrations. A different supported convention needs its own formula, fee order, rounding/domain rules and independent vectors; naming a math profile or providing an opaque server quote is insufficient. Proof sketches in Section 12 apply only under their stated formulas and premises.

6.1 Domain and virtual balances

The reference math sets virtual assets $V_a = 1$ and immutable positive virtual shares V_s. Define adjusted balances $X = A + V_a$ and $Y = S + V_s$. Virtual balances are constants in the conversion function, not property that can be spent.

The candidate bounded-integer profile permits individual stored economic quantities up to $2^{63} - 1$, subject to any stricter ledger bound. Intermediate arithmetic MUST be exact and sufficiently wide; implementations must not use floating-point arithmetic. Aggregate quantities are checked before serialization and execution. Unsupported versions and invalid inputs return an explicit error, not a usable zero quote.

OpenZeppelin documents virtual-balance and rounding defenses for vault share conversion. CTVS adopts the repository's named arithmetic profile, while authenticating custody through UTxO transitions rather than external account balances. [11], [1]

6.2 Conversion functions

For a nonnegative asset quantity a and share quantity q:

$$\text{sharesDown}(a) = \text{floor}\left(\frac{aY}{X}\right), \qquad \text{sharesUp}(a) = \text{ceil}\left(\frac{aY}{X}\right). \tag{7}$$

$$\text{assetsDown}(q) = \text{floor}\left(\frac{qX}{Y}\right), \qquad \text{assetsUp}(q) = \text{ceil}\left(\frac{qX}{Y}\right). \tag{8}$$

Fee-free conversion views use the down versions. Direct previews use the action-specific formula below. The exact fixed-state preview equals the validated economic result; later states do not inherit that guarantee. Output minima and input maxima must be enforced in the signed transaction.

The adjusted price $p = X/Y$ is a pricing convention, not an oracle or an unconditional redeemable-value promise. At zero supply, display a bootstrap/residual status rather than dividing physical backing by zero.

6.3 Entry and exit fees

Let $B = 10{,}000$, with entry rate b_e and exit rate b_x satisfying $0 \le b < B$. The reference fee is a markup on the pre-fee amount; its fee-inclusive inverse is not the same as a percentage-of-gross convention:

$$\text{feeRaw}(x, b) = \text{ceil}\left(\frac{xb}{B}\right), \qquad \text{feeTotal}(x, b) = \text{ceil}\left(\frac{xb}{B + b}\right). \tag{9}$$

For example, a gross deposit of 101,000 with a 100-basis-point entry markup pays 1,000 in fees and admits 100,000 as backing. A 1% fee charged directly on gross would be a different profile.

Operation	Computation	User protection
Deposit gross d	$f = \text{feeTotal}(d, b_e)$; $n = d - f$; $q = \text{sharesDown}(n)$	$q \ge q_{min}$
Mint exact q	$n = \text{assetsUp}(q)$; $f = \text{feeRaw}(n, b_e)$; $d = n + f$	$d \le d_{max}$
Withdraw net w	$f = \text{feeRaw}(w, b_x)$; $g = w + f$; $q = \text{sharesUp}(g)$	$q \le q_{max}$
Redeem exact q	$g = \text{assetsDown}(q)$; $f = \text{feeTotal}(g, b_x)$; $w = g - f$	$w \ge w_{min}$

The reference execution profile rejects zero economic input, zero shares issued, or zero net redemption output. Fee-free conversion of zero remains a valid zero view. Fees, limits, liquidity, and authorization are different checks; a mathematically computed preview is not evidence that the transaction is executable.

6.4 Limits and the corrected maximum-operation algorithm

Let A_{cap} be the optional immutable backing cap. Admission requires $A + n \le A_{cap}$. Exits require sufficient participating shares, gross backing, and physical liquidity after preserving fees/storage. A positive-output requirement creates a *lower* execution threshold, while caps and integer domains create upper thresholds. The whole execution predicate is therefore not necessarily prefix-monotone.

At $A = S = 0$, $V_s = 1$, entry markup 9,999 basis points and backing cap one:

Gross deposit	Fee	Net backing / shares	Execution
1	1	0 / 0	Reject: zero issuance
2	1	1 / 1	Accept economically
3	2	1 / 1	Accept economically
4	2	2 / 2	Reject: cap

A last-true binary search over the entire acceptance predicate can incorrectly return zero. The corrected algorithm searches only the monotonic **upper-resource predicate**: backing cap, supply bound, fee-liability bound, and permitted model value bounds. It then checks positive issuance at the upper candidate. [22]

```
max_deposit(state, terms):
    reject invalid state/profile; return unavailable for disabled/paused entry
    hi = bounded gross-amount domain maximum
    d = largest x in [0, hi] satisfying upper_capacity(x)
        # binary search requires upper_capacity(0) = true
        # and upper_capacity must be prefix-monotone
    if d == 0 or deposit_shares(d) == 0: return known_zero
    return economic_maximum(d)
```

Do not call the returned amount a resource-feasible transaction maximum. The search does not measure serialized size, execution units, input availability, or minimum ADA. Maxima for other operations require their own proved monotonic upper constraints; a generic binary search over arbitrary feasibility predicates is not specified. Expose minimum economic size, economic maximum, construction limitations, unsupported, stale, paused, and unknown liquidity separately.

In this reference profile, nonzero state-action amounts and user bounds are positive integers; views can still convert zero. An immutable cap, a pause, or a disabled execution mode does not change the conversion equation. The economic maximum-deposit implementation agrees with exhaustive enumeration in 384 small configurations; no unbounded or ledger-resource conclusion follows.

6.5 Units and rate scaling

Common unit contract. Inputs and outputs MUST remain integer quantities of their identified assets. No common 18-decimal scale is required. Providers MAY use another internal representation, but MUST convert to the advertised ledger units before asserting a result and MUST publish every scale and intermediate rounding rule. Different output assets cannot be added merely by aligning decimals; converting them into one valuation asset requires its own authenticated exchange-rate basis.

For a native proportional conversion, let q be receipt units, B recognized backing in underlying units and T participating receipt supply in the same receipt units as q. Then:

$$a = \text{floor}\left(\frac{qB}{T}\right), \qquad T > 0. \tag{10}$$

Receipt units cancel; this formula needs no additional decimal factor. It illustrates a native provider, not a replacement for Sections 6.1–6.3's virtualized reference equations or a universal treatment of fees and losses.

For eight-decimal receipts and six-decimal underlying, assume 1,000,000 whole receipts backed by 1,100,000 whole underlying tokens:

```
q =        50,000,000,000 receipt units       (500 receipts)
B =     1,100,000,000,000 underlying units
T =   100,000,000,000,000 receipt units
a = floor(q * B / T) = 550,000,000 underlying units
```

The result displays as 550 underlying tokens. Display formatting does not participate in this calculation.

A different source may report $r/10^{d_r}$ whole underlying tokens per whole receipt. If its authenticated unit interpretation defines d_s receipt decimals and d_a underlying decimals, the down-rounded conversion is:

$$a = \text{floor}\left(\frac{q\, r\, 10^{d_a}}{10^{d_s}\, 10^{d_r}}\right). \tag{11}$$

For $r = 110{,}000{,}000$, $d_r = 8$, $d_s = 8$ and $d_a = 6$, the same input produces 550,000,000 underlying units. The provider MUST authenticate the rate's direction, asset pair and scale through pinned configuration or verified evidence. A signed source MUST bind these meanings through its authenticated schema or report. A builder cannot independently select a more favorable exponent.

Multiply with exact intermediate arithmetic before the specified division. Exact-output requirements round up under their declared action rule; conversions and exact-input outputs round down. Source quantization and any unavoidable intermediate rounding MUST be disclosed. Results exceeding the supported quantity domain fail explicitly; normalizing to a larger internal scale does not enlarge that domain. Zero views and zero-output execution retain their distinct Section 6 rules.

6.6 Verified pricing assertions

Candidate optional interface. A builder supplies `Price` as the redeemer of an approved pricing withdrawal validator. The provider authenticates its evidence and verifies every assertion. A consumer requires that exact provider invocation, reads the same redeemer and selects the exact subject, operation and fixed input. There is no cross-script return value and no mutable price store implied by this interface.

Assertion	Fixed input	Claimed result	Result rounding
`ConvertToAssets`	Share units	Underlying units, excluding operation entry/exit fees	Down
`ConvertToShares`	Underlying units	Share units, excluding operation entry/exit fees	Down
`PreviewDeposit`	Gross underlying input	Shares issued after declared operation fees	Down
`PreviewMint`	Shares required	Gross underlying input including declared operation fees	Up
`PreviewWithdraw`	Net underlying required	Shares surrendered including declared operation fees	Up
`PreviewRedeem`	Shares surrendered	Net underlying after declared operation fees	Down

Previews exclude wallet network fees and non-economic storage ADA. They do not reserve liquidity or guarantee construction, permission, inclusion or future execution. The candidate is context-free; owner-dependent pricing requires a separately specified authenticated context and consumer binding. Unsupported methods fail, including unsupported assertions accompanying a valid one.

The provider MUST match the market and asset pair, authenticate native state or the declared attestation, apply its published accounting and fee rules, enforce applicable freshness/state-basis requirements, and reject every incorrect assertion. A signature establishes reporter authorization under an attested profile, not independent economic truth. Structural decoding alone validates none of these properties.

The consumer MUST require the configured provider's script credential with withdrawal amount zero, resolve its redeemer by withdrawal purpose, match market/assets and select exactly one matching query. It MUST separately

enforce custody, liabilities, supply, user bounds and destinations. A provider's success does not authorize moving receipts or establish ownership of them. Both validators must pass in the same transaction; execution order is not a data dependency.

Accounting evidence and code witnesses are distinct. An evidence pointer names a state output that the transaction must include as a reference input. The provider reads that output from the transaction context; it does not fetch an API or execute the evidence. Compiled provider code is supplied as a witness or by a separate reference-script UTxO. Referencing that code does not invoke it; the withdrawal purpose does. Read-only reference inputs remain unspent. A recipe consuming native state must separately define whether assertions use its pre-transition, post-transition or reconciled basis.

Worked claim. The Section 6.5 example asserts `ConvertToAssets { shares: 50000000000, assets: 550000000 }`. The provider verifies the native backing and supply. A receipt-holding consumer verifies that it owns the claimed quantity and uses the value in its permitted accounting transition. An optional successor checkpoint records that transition's valuation; it is not perpetual evidence of the current rate.

Appendix B defines the exact candidate encoding. The shared decoder and hypothetical LendingA example are local implementation material, separate from the pinned Direct Custody Profile evidence. The example is a valuation checkpoint, not a complete vault, signature verifier or deployed protocol integration.

The Direct Custody Profile still rejects withdrawals under Sections 3.4 and 8.2. A deployment adopting this mechanism MUST define a separate reviewed composition profile with approved withdrawal purposes, witness requirements and complete allocation checks across validators, planners and pre-signing effects. Appending a zero withdrawal to an existing reference transaction does not enable the extension.

6.7 Complete Price construction and decoding example

Isolated LendingA binding. This example values a receipt position, not a complete managed vault. Its published binding verifies all six conversion and preview methods against authenticated native accounting, with no operation fees or time-dependent accrual. Exact-input results round down; exact-output costs round up. Zero supply rejects. At zero backing, share-to-asset conversion and redemption preview report zero value; the other four methods reject. Signed reports, bootstrap and actual deposit/redemption execution remain unsupported. The updated provider has compiled and signed local tests; the retained node trace applies to the earlier conversion-only program hash. A provider publishing different rules must specify and verify them independently.

Assume the consumer holds 500 receipts with eight display decimals. The native market recognizes 1,100,000 underlying tokens with six decimals against 1,000,000 participating receipts. The transaction builder reads ledger integers, computes the Section 6.5 result, and proposes 550 underlying tokens. The provider's configured NFT identifies the market; token labels and display decimals do not authenticate it.

Transaction component	Contents and purpose	Consumed?
Receipt input	Consumer address, 50,000,000,000 receipt units, storage ADA, previous checkpoint	Yes
Wallet inputs	Fees, required collateral and ordinary change funding	Funding is spent; collateral follows ledger rules
Native state reference	Configured script address, exactly one market NFT, inline backing 1,100,000,000,000 and supply 100,000,000,000,000	No
Program references, if used	Actual compiled provider and consumer programs, each checked against its applied script hash	No

Transaction component	Contents and purpose	Consumed?
Withdrawal	Script credential of approved provider, amount zero, `Price` redeemer	Invokes provider; moves no rewards
Receipt successor	Same consumer address and full receipt-input value; checkpoint 550,000,000	Created

The builder requires chain-data access to discover the current unspent NFT-bearing market output and receipt input. It can use a node/indexer provider, a wallet backend or another verified chain-data source. Running a dedicated indexer or using WASM is not required by the interface. A code-reference location is an availability hint; its program must hash to the approved identity. The evidence reference is a full transaction ID and output index, not a position in the reference-input list.

The shared TypeScript codec constructs the common claim. In abbreviated code, with assets and references already authenticated:

```
const claim = {
  version: 1n,
  marketId, shareAsset, underlyingAsset,
  assertions: [{
    kind: "ConvertToAssets",
    shares: 50_000_000_000n,
    assets: 550_000_000n,
  }],
  evidence: constr(0, [outRefData(marketStateRef)]),
};
const redeemer = hex(encodeData(priceToData(claim)));
const decoded = priceFromData(decodeData(redeemer));
```

Decoding requires constructor 0 with exactly six fields, encoded version 1, exact asset encodings, 1..16 assertions, supported constructor shapes and bounded evidence. The conversion is constructor 0 with two integers, input first and result second. Duplicate queries reject even when their results agree. These checks establish well-formed Data, not a truthful conversion.

The LendingA provider then decodes `OnChain` as constructor 0 containing one exact `OutputReference`. It selects exactly that included reference input, checks the configured address and NFT, decodes its exact two-integer inline native datum, and verifies **every** assertion:

```
assert declared market and asset identities match Config
assert backing >= 0 and receipt supply > 0
for each assertion:
    require ConvertToAssets
    require assets == floor(shares * backing / receipt_supply)
```

The consumer separately requires the approved script's zero withdrawal and reads its redeemer by semantic withdrawal purpose. It matches market and both assets, then selects exactly the conversion for its actual held receipt quantity. It requires one consumed position and one successor at its address, the complete input value preserved, no successor reference script, and the exact recorded value. A lie of 700 underlying tokens may pass consumer

selection and custody checks but fails the provider's arithmetic; the ledger requires both invoked validators to pass. No execution ordering or cross-script return value is involved.

The signed construction fixture uses actual parameterized programs and derived hashes. It registers the reward credential, mints issuer-controlled test assets, publishes real reference programs, balances and signs the transaction, re-evaluates final CBOR for both purposes and reads the accepted local successor. Its two paths supply programs as witnesses or references. The issuer-controlled policies do not establish production NFT uniqueness or native transition correctness. The source protocol remains responsible for those guarantees and for accounting after applicable liabilities.

A market output spent before submission is unavailable evidence. Rebuild from its replacement and recalculate; do not silently reuse the old reference or checkpoint. A later tend transaction likewise uses its own evidence and actual outputs. A deposit priced before tend is a share of the vault's current recognized assets and liabilities, including idle funds; a hypothetical future receipt count is not an asset already held. Later gains and execution losses affect participating shareholders through the declared accounting rules. A conversion never reserves liquidity or establishes the feasibility of an unwind.

Acceptance boundary. The shared byte vector checks TypeScript/Aiken agreement for all six assertion constructors. Compiled handler and signed local tests exercise this provider's mixed-unit conversion, rejection paths and resource costs, including 16 distinct assertions. This is one hypothetical binding. An opt-in test additionally submitted and read back both paths and a 16-assertion reference invocation on an isolated magic-42 Cardano node through Kupo/Ogmios; the dated validation record retains transaction bytes and costs. This closes the isolated example's positive node-acceptance check only. Public deployment, independent implementations, negative node submission, signed-report verification and managed composition remain distinct gates.

7. Direct transitions and authorization

7.1 Accounting deltas

Every successful state transition verifies the old value equation, recomputes the action, checks the successor equation, and increments the state sequence. The variables below use the quotes from Section 6.

Action	A'	F'	S'	Physical underlying change
Deposit / mint	$A + n$	$F + f$	$S + q$	$+d$
Withdraw / redeem	$A - g$	$F + f$	$S - q$	$-w$
Collect all fees	A	0	S	$-F$
Top up reserve by t	A	F	S	ADA reserve $+t$ only
Change pause flags	A	F	S	None

For exits, A falls by gross value g, even though physical custody falls only by w. The difference has become a fee liability. Removing w from backing and also accruing f would incorrectly overstate shareholder assets.

7.2 Authorization and key-only reference roles

Common obligation. Prove authorization of the intended action and committed value; do not prescribe one universal authorization language. A native implementation may use an established script guard, provided its execution is bound to the vault/request, action, applicable terms, amount constraints and permitted destinations. Guard identity or a reference to its code is not execution. The Direct Custody Profile remains key-only as specified below.

The direct action spends ordinary funding/share inputs under their own ledger authorization. It fixes receiver, amount, and limit in a StateRedeemer. There is no global native-token allowance and no implicit caller. In Direct Custody Profile the remaining funding/share inputs are key-controlled; arbitrary script-controlled direct input composition is not enabled.

A configured `pause_key: Some(k)` authorizes SetPause only when k is an explicit required signatory with a valid ledger witness. `None` disables that privileged branch. The reference profile has no capability slot in Terms and no always-true script-authority approximation. A controller capability is a separate profile requiring executed authorization, replay/domain binding, its own protected successor, and compatibility with output allocation.

A protocol can create a request under its own spending rules and name a supported key controller and script destination. That does not establish a native script-controlled cancellation implementation. Likewise, a destination's script is not executed merely when an output is created there. Local key-role and destination tests must not be described as capability or receiving-validator execution. [14], [12]

7.3 Pause, fees, and reserves

Entry and exit pause flags are separate. An entry pause blocks deposit and mint; an exit pause blocks withdraw and redeem. Only the declared authority can change flags. State-reserve top-up and correct fee collection remain available. CTVS-2 separately defines that cancellation and funded-claim delivery are not blocked by these flags.

Fee collection is permissionless but pays only the immutable destination and exact economic amount. The collector supplies any additional output ADA and network fee. The transaction cannot change backing or share supply. A zero fee balance is not a valid collection action.

State-reserve top-up accepts only a positive increase and cannot modify other accounting fields. There is no repayment right for the contributor and no reserve withdrawal in the reference profile.

7.4 Full transition obligations and mint branch coupling

Before applying the arithmetic, check profile/mode, the selected State action tag, authentic own input and config, all old state bounds/value, permitted transaction family, and valid output indices. Recompute the quote and user bound; create one exact successor and a distinct protected payout. Direct entry is enabled by bit 1, direct exit by bit 2. SetPause alters only pause flags and sequence; TopUpReserve alters only storage and sequence. All accounting transitions increment sequence once.

StateSpend and MintPurpose are different redeemer envelopes. A nonzero SHARE mint/burn must use the supply-update action pointing to the actual consumed state; that state must independently verify its exact supply delta. The state MUST NOT assume a minting policy runs for a zero-net batch. No ID/STATE recreation, unrelated mint, or alternative share-burn route is permitted. Genesis alone creates the two identities. Exact envelope tags and action arities appear in Section 10; the asynchronous gross obligations appear in CTVS-2 Section 6. [12]

7.5 Script-controlled user acceptance target

A real protocol-controlled user is an early interoperability requirement, not evidence already supplied by key-destination tests. Pin one treasury/portfolio guard and demonstrate: authorized funding of the supported deposit or request; successful output to the committed protocol destination/datum; cancellation of still-pending property back to the committed refund; and rejection of an operator-selected replacement recipient. Then spend the receiving output under its real validator to establish usability, not merely address equality.

Use the native guard's actual mechanism; CTVS does not require a new capability NFT or permission expression tree. Record all purposes the composition invokes and prove complete non-overlapping obligations. Such a transaction

must not claim Direct Custody Profile restricted-input conformance when it consumes a foreign script or uses a withdrawal hook. Required compiled authorization, ledger and independent-builder evidence remain open. [22]

8. Payment attribution and reference composition

8.1 Obligations, not address searches

The common requirement is no double satisfaction of protected obligations. A native implementation may use disjoint outputs or a proved aggregate allocation. For aggregate quantities $x_{b,o,a}$ attributed to obligation b, output o and asset a, complete fulfillment and output capacity must both hold: $\sum_o x_{b,o,a} = q_{b,a}$ and $\sum_b x_{b,o,a} \leq V_o(a)$, with matching destination conditions. The allocation is authenticated by actual validators, not an off-chain transcript. If a shared input serves several users, its per-intent protected partitions also require an authenticated, exhaustive accounting rule.

The Direct Custody Profile chooses the simpler disjoint-output mechanism below. Relaxing its purpose restrictions without another demonstrated coverage proof is not an allowed editorial change.

A payment obligation includes its source operation, asset, economic quantity, exact destination, datum condition, and storage/refund requirements. Two obligations to the same destination cannot each count the same value independently.

Let $\mathcal{B}$ be the set of obligations. An allocation function maps each obligation to an output index. The reference rule requires distinct outputs for distinct protected obligations:

$$a : \mathcal{B} \to \{0, \ldots, |O| - 1\}, \qquad b_1 \neq b_2 \Rightarrow a(b_1) \neq a(b_2). \tag{12}$$

Each selected output must satisfy its obligation. State continuation, user payment, and fee payment cannot share an output. Output indices are permitted as validated lookup hints; identity does not depend on an unverified array position.

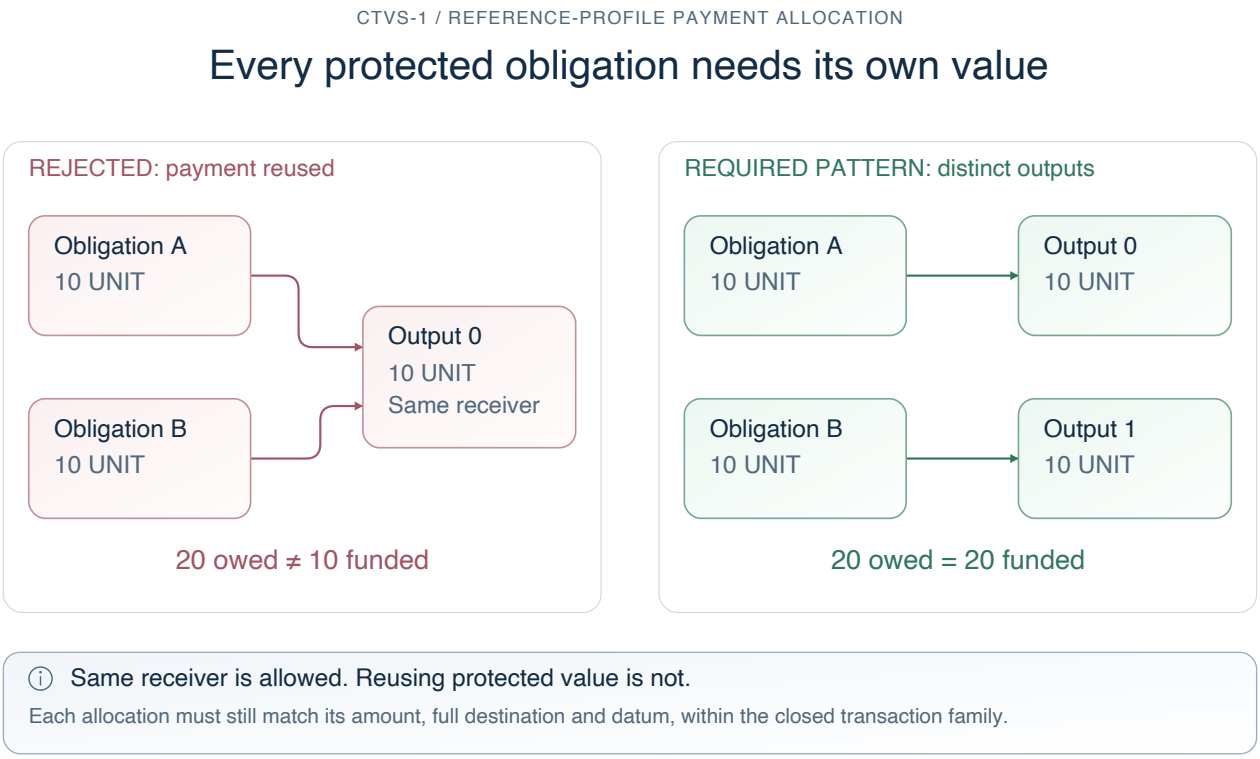


Figure 4: *Protected obligations need exclusive output allocation. Same receiver is permitted, but one ten-unit payment cannot satisfy two separate ten-unit obligations. Amount, full destination, datum, and closed transaction domain still require validation.*

8.2 Restricted composition inputs and purposes

Local output-index injectivity cannot prevent an unrelated validator from counting the same payout. The Direct Custody Profile therefore validates the transaction's entire permitted script-input family. A direct action has one authentic state plus key-controlled funding/share inputs. Batch extends that set only with this vault's request inputs. No other vault state, foreign script input, simultaneous claim delivery, or capability input is admitted.

Withdrawals, including zero-withdrawal hooks, certificates, votes, proposals, treasury declarations, donations, unrelated minting, and unsupported purposes are rejected as specified in Section 3.4. These are validator requirements, not builder conventions. Reference-only data does not fund payouts. Config/state and protected destination outputs forbid attached reference scripts.

The restricted family closes the reference proof's script-protected obligation set; it intentionally limits atomic composition. A shared allocation verifier or reviewed adapter is required before relaxing it. Request creation from another protocol and delivery to a script destination remain distinct from consuming that protocol's script during settlement. [12], [14]

Section 6.6's optional `Price` invocation requires a separate composition profile. It does not relax this profile's empty-withdrawal rule or transfer its restricted-family allocation proof to arbitrary script composition.

8.3 Destinations

A destination fixes the full ledger address and either no datum or an exact inline datum. Datum-hash-only destinations are outside this profile. The validator does not replace a recipient protocol's datum with a mandatory CTVS receipt.

For a native-token payout, the protected output contains exactly the economic quantity of the underlying. ADA additionally funds its storage. For an ADA payout, the output contains the economic amount plus any explicitly returned reserve/top-up. Allocation and closed balance equations establish that a minimum-output top-up did not reduce another protected payment.

8.4 Destination and recovery preservation are different from economic quoting

Destination matching checks both credentials, exact datum mode/value, and no attached reference script. A native economic payout fixes its economic token quantity; ADA top-ups may be external. For CTVS-2 fixed refunds and deliveries, the stronger rule is componentwise preservation of the **entire actual source value**, not just a declared amount:

$$\text{DestinationMatches}(o, c) \ \wedge \ \forall a, \ V(o)[a] \geq V(c)[a]. \tag{13}$$

Distinct claims/refunds retain disjoint allocations. The exact ledger address subset and optional-datum representation are given in Section 10. Recipient data cannot be overwritten by a CTVS receipt; the separate claim object carries the source references. [12]

9. Worked UTxO transactions

9.1 Example conventions

These are complete economic/value sketches, not serialized ledger transactions. `ID`, `STATE`, and `SHARE` abbreviate distinct authenticated assets; `UNIT` is a hypothetical native underlying. Numeric quantities are base units. The illustrated 3,000,000-lovelace state reserve, 2,000,000-lovelace destination reserve, and 400,000-lovelace transaction fee are assumptions, not current network minima or fee estimates. A builder must calculate actual sizes, fees, collateral, and witnesses. Reference inputs do not contribute spendable value.

Examples 1, 2, and 4 form one accounting sequence with $V_a = V_s = 1$ and 100-basis-point entry/exit markup. A previously authenticated initial state is assumed. State/configuration inputs and destinations use their required inline datums; ordinary key outputs below have no datum.

9.2 Example 1 - native-token deposit

Start with $A = S = 1{,}000{,}000$, $F = 0$, and $R = 3{,}000{,}000$. Gross deposit $d = 101{,}000$ gives $f = 1{,}000$, $n = 100{,}000$, and $q = 100{,}000$. The user requires at least 99,900 shares.

Side	Object	Value / economic data
Reference	Config `cfg#0`	1 ID; immutable terms
Input	State `s0#0`	1 STATE; 1,000,000 UNIT; 3,000,000 lovelace
Input	User `u0#0`	101,000 UNIT; 6,000,000 lovelace
Mint	SHARE	+100,000

Side	Object	Value / economic data
Output	Successor state	1 STATE; 1,101,000 UNIT; 3,000,000 lovelace
Output	Receiver	100,000 SHARE; 2,000,000 lovelace
Output	User change	3,600,000 lovelace
Fee	Network	400,000 lovelace

The successor datum has $A' = 1{,}100{,}000$, $F' = 1{,}000$, and $S' = 1{,}100{,}000$. Native asset balance is $1{,}000{,}000 + 101{,}000 = 1{,}101{,}000$. ADA balance is $9{,}000{,}000 = 3{,}000{,}000 + 2{,}000{,}000 + 3{,}600{,}000 + 400{,}000$.

9.3 Example 2 - redemption with an accrued exit fee

Use Example 1's successor. Redeem 50,500 shares with a minimum payout of 49,900 UNIT. Since the adjusted exchange rate remains one, gross redemption is 50,500, fee is 500, and net payout is 50,000.

Side	Object	Value / economic data
Input	State	1 STATE; 1,101,000 UNIT; 3,000,000 lovelace
Input	Share holder	50,500 SHARE; 5,000,000 lovelace
Mint	SHARE	-50,500
Output	Successor state	1 STATE; 1,051,000 UNIT; 3,000,000 lovelace
Output	Receiver	50,000 UNIT; 2,000,000 lovelace
Output	User change	2,600,000 lovelace
Fee	Network	400,000 lovelace

The successor is $A' = 1{,}049{,}500$, $F' = 1{,}500$, $S' = 1{,}049{,}500$. Therefore $A' + F' = 1{,}051{,}000$, exactly the remaining physical UNIT balance. The 500-unit exit fee was not paid out and does not remain share backing.

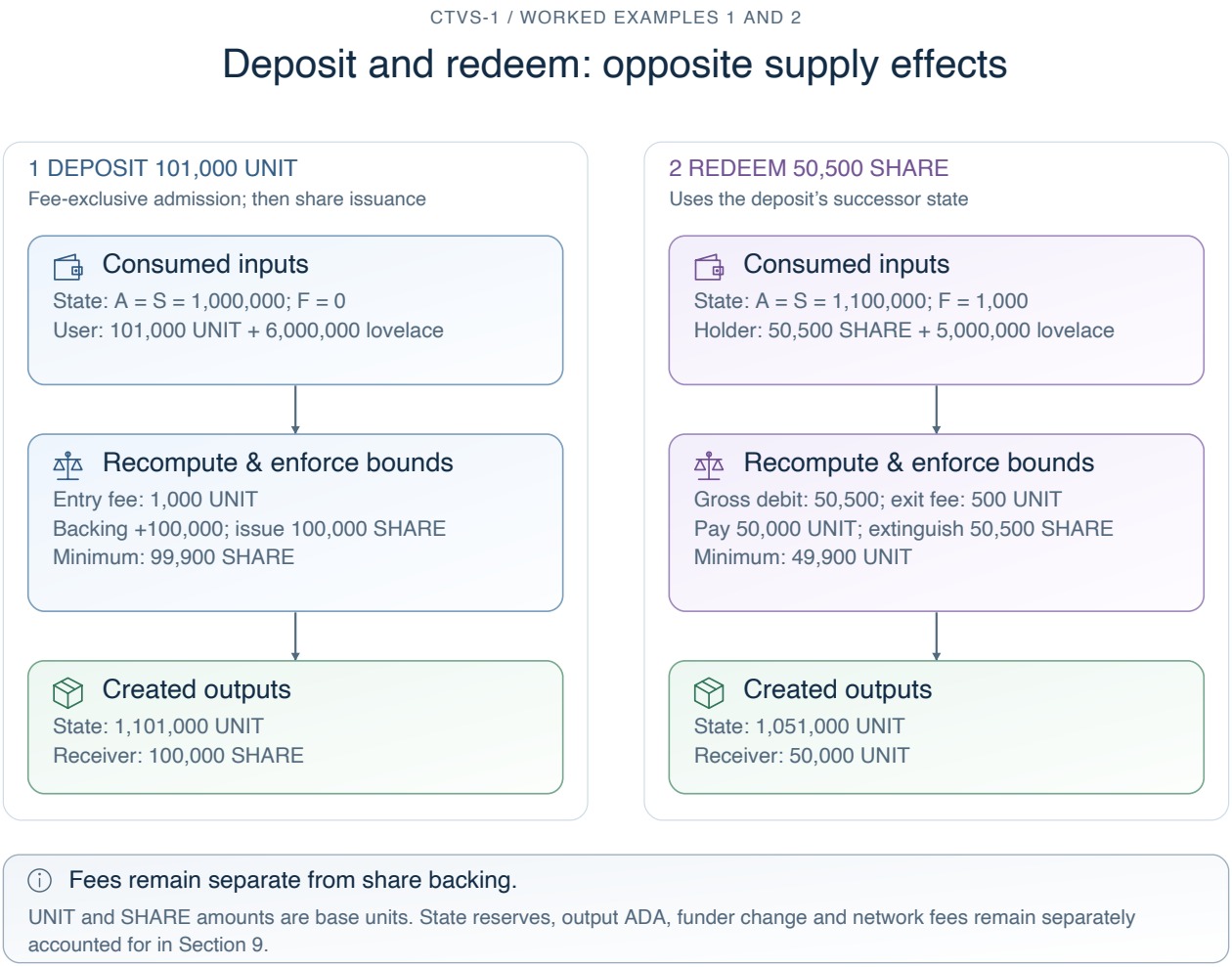


Figure 5: *Direct deposit and redeem use the same authenticated State boundary but have opposite supply effects. The figure compares fee-exclusive admission and share issuance with gross backing debit, fee accrual, share extinction, and net payout.*

9.4 Example 3 - ADA underlying without reserve dilution

Independently start at $A = S = 100{,}000{,}000$, $F = 0$, $R = 3{,}000{,}000$, with $V_a = V_s = 1$. Deposit 10,100,000 lovelace economically. The fee is 100,000 and shares issued are 10,000,000.

Side	Object	Value
Input	State	1 STATE; 103,000,000 lovelace
Input	User	15,100,000 lovelace
Mint	SHARE	+10,000,000
Output	Successor state	1 STATE; 113,100,000 lovelace
Output	Receiver	10,000,000 SHARE; 2,000,000 lovelace
Output	Change	2,600,000 lovelace
Fee	Network	400,000 lovelace

The state now separates 110,000,000 backing, 100,000 fees, and 3,000,000 reserve. The user's 2,000,000-lovelace share-output reserve does not buy additional shares. Total ADA is 118,100,000 on both sides including the fee.

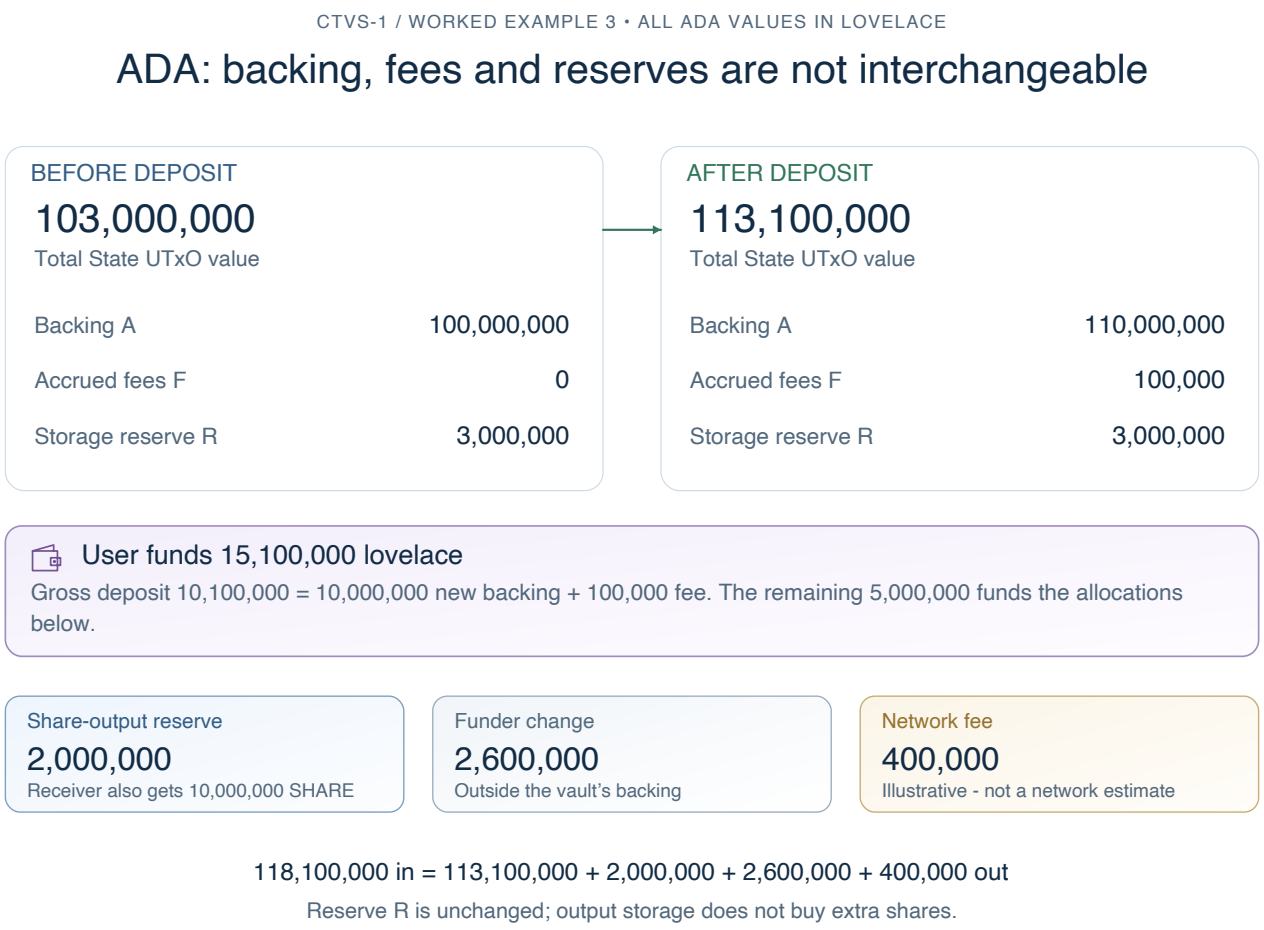


Figure 6: *ADA accounting keeps backing, accrued fees, and State storage separate. The partition boxes are not proportional; their purpose is to show that output storage, change, and network fees do not purchase shares or become backing.*

9.5 Example 4 - collecting the fees from Example 2

The state contains 1,051,000 UNIT, comprising 1,049,500 backing and 1,500 fees. An external collector contributes 5,000,000 lovelace. The successor keeps 1,049,500 UNIT and its 3,000,000-lovelace reserve. The configured treasury receives 1,500 UNIT plus 2,000,000 lovelace. Collector change is 2,600,000 and the fee is 400,000.

No SHARE is minted or burned. A and S are unchanged; $F' = 0$. A payout to the correct treasury address with the wrong inline datum is invalid.

9.6 Example 5 - residual backing after the last redemption

This is a minimal arithmetic illustration, with storage omitted. Set $V_a = 1$, $V_s = 2$, and no fees. From $A = S = 0$, mint three shares for $\text{ceil}(3/2) = 2$ assets. Redeeming all three returns $\text{floor}(3(2+1)/(3+2)) = 1$ asset. The final state is $A = 1, S = 0$.

The reference policy preserves this residual and applies the same immutable math to future entry. It does not distribute an extra unit to the last redeemer, reset balances, or permit a sweep. The permanent-state profile remains open or paused. Any different residual disposition is a separately disclosed economic profile.

9.7 Exact datum and direct-action correspondence

Use Example 1 with old sequence 7 and no pause. Let `p` be the authenticated vault-policy bytes and `hK` the deployment's terms commitment; these symbolic names are not executable hashes. In the exact constructor notation of Section 10:

```
old_state = C(0,[CTVS,2,p,hK,7,
                 1000000,1000000,0,3000000,0])
new_state = C(0,[CTVS,2,p,hK,8,
                 1100000,1100000,1000,3000000,0])
StateRedeemer = C(0,[CTVS,2,
    C(0,[receiver,101000,99900,0,1])])
MintRedeemer = C(3,[CTVS,2,C(1,[consumed_state_ref])])
```

The example fixes successor output 0, receiver output 1, and key change output 2. State field order is backing, supply, then accrued fees. The receiver is the full Destination constructor, not a string. The normal mint map is SHARE(+100,000); the reference config and state token are independently authenticated. A transaction with these numeric deltas but a wrong datum, wrong state branch, substituted config, or reused payout index is invalid under the candidate predicates.

Examples 2 and 4 advance the same sequence for redemption and fee collection. Their inner State action tags are respectively 3 and 5; no free-standing checkpoint field is added to the successor. Example 3 uses the same ten-field State structure with ADA represented by Terms.Asset constructor 0. These are candidate construction sketches, not serialized accepted transactions. [12], [13]

10. Datum, redeemer, and encoding specification

Normative for the existing reference wire. The exact constructors, counts, bounds, names and commitment bytes in Section 10 define the Direct Custody Profile. They are not a mandatory migration format for every native integration. Native mappings must publish equally precise decoding and authentication rules for their own implementation and satisfy Section 2.5. The four files under `candidate-wire/` specify the exact reference encoding and fixtures.

10.1 Candidate identity, notation, and recursive primitives

`C(t,[...])` is Plutus `Constr t fields`, not a CBOR major type and not a JSON object. `B[n]` is a byte string of exactly n bytes. `Q` is an integer in 0 through 2^63-1; `Q+` is positive in that range. An output index is a candidate integer in 0 through 65,535, additionally checked against the actual transaction length. This bound is not a capacity promise.

The magic bytes are ASCII `CTVS` (`43545653` hex). The candidate wire-version integer is 2. Amounts in the JSON fixture representation are exact decimal strings, not floating-point values.

```
Credential = C(0,[B[28]])                -- verification key hash
           | C(1,[B[28]])                -- script hash

Option<T>  = C(0,[T])                    -- Some
           | C(1,[])                     -- None

StakeRef   = C(0,[Credential])           -- inline credential
Address    = C(0,[Credential, Option<StakeRef>])
Destination= C(0,[Address, Option<BoundedData>])

Asset      = C(0,[])                     -- ADA
           | C(1,[B[28], Bytes[0..32]])    -- native policy and name
OutRef     = C(0,[B[32], output_index])
Controller = C(0,[B[28]])                -- key profile only
Settlers   = C(0,[])                     -- any signing reward key
           | C(1,[List<B[28]>])          -- sorted, unique, 1..16 keys
```

Address nesting follows the selected Aiken standard-library representation for the supported subset. Pointer staking addresses and bootstrap addresses are excluded here, not silently interpreted as enterprise addresses. All payment and staking credentials are compared. On-chain address data does not itself contain the user-facing network identifier. The network-domain commitment is a deployment convention; clients and ledger construction must verify the actual network rather than pretending the validator has an implicit chain-ID oracle. [14], [15]

A destination with `None` requires `NoDatum`. `Some(d)` requires an inline datum structurally equal to d, not a datum hash and not just any datum at the right address. `NoDatum`, `DatumHash`, and `InlineDatum` are distinct ledger variants. [14]

10.2 Immutable Terms and Config

```
Terms = C(0,[
  profile,                 -- exactly 0: this reference combination
  network_domain,          -- B[32], configured chain-domain fingerprint
  underlying,              -- Asset
  virtual_shares,          -- Q+; virtual assets are fixed at 1
  max_backing,             -- Option<Q>
  entry_bps, exit_bps,      -- integers 0..9999
  fee_destination,         -- Destination
  pause_key,               -- Option<B[28]>
  settlers,                -- Settlers
  execution_modes,         -- integer 1..15
  max_batch,               -- integer 1..16, provisional structural cap
  descriptor_hash          -- Option<B[32]>, non-executable metadata
])

Config = C(3,[
  magic, wire_version, vault_policy, seed,
  terms_hash, terms, storage_lovelace
])
```

`Config` has exactly seven fields; `Terms` has exactly thirteen. Mode bits are direct entry 1, direct exit 2, asynchronous entry 4, and asynchronous exit 8. Direct entry includes deposit and mint; direct exit includes withdraw and redeem. Unknown bits are not ignored. The descriptor is not executable builder code and is not an alternative authority for custody.

Verify the terms commitment against the program's authenticated parameter, not only against another field in the same untrusted datum. Config storage is positive. Config value is exactly its reserve plus ID(1), with no reference script.

10.3 Exact accounting State

```
State = C(0,[
  magic, wire_version, vault_policy, terms_hash,
  sequence, backing_assets, economic_supply, accrued_fees,
  storage_lovelace, pause_flags
])
```

The State has exactly ten fields. No unconstrained `extensions: Data`, mutable asset identifier, mutable fee destination, or generic oracle field is included. `pause_flags` is 0..3, with entry bit 1 and exit bit 2. All nonnegative quantities and aggregate physical balances are checked; individually bounded fields do not prove their sum fits.

`sequence` increments exactly once per accounting transition and has no authority over unrelated UTxOs. The direct profile has no meaningful NAV epoch, so no free-standing epoch field can be arbitrarily changed. A managed state needs a separate exact profile, not hidden managed semantics in this State.

The closed value equations are:

$$V_{\text{state}}(\text{ADA}) = A + F_{\text{fees}} + R \quad \text{for ADA underlying,} \tag{14}$$

$$V_{\text{state}}(u) = A + F_{\text{fees}}, \qquad V_{\text{state}}(\text{ADA}) = R \quad \text{for native-token underlying.} \tag{15}$$

STATE(1) is also present, and every other native asset is absent. No reference script or staking credential is attached. Only an explicitly authorized reserve-top-up action increases R; ordinary operations preserve it. Fees and reserves cannot fund the network fee by an implicit state reduction.

10.4 Redeemer envelopes and action constructors

Every action envelope has exactly `[magic, wire_version, action]` and a role tag: StateSpend 0, RequestSpend 1, ClaimSpend 2, MintPurpose 3. A decoder for one role does not accept another role's tag. Constructor tags are scoped to the expected type; tag zero does not globally mean the same operation.

Inner state action tag	Fields, in order
0 Deposit	receiver, gross_assets, min_shares, state_output, receiver_output
1 Mint	receiver, target_shares, max_assets, state_output, receiver_output
2 Withdraw	receiver, net_assets, max_shares, state_output, receiver_output
3 Redeem	receiver, shares, min_assets, state_output, receiver_output

Inner state action tag	Fields, in order
4 Batch	state_output, entries, reward_key, optional_reward_output
5 CollectFees	state_output, fee_output
6 TopUpReserve	additional_lovelace, state_output
7 SetPause	pause_flags, state_output

A `SettleEntry` is `C(0,[request_ref,claim_output,claim_topup])`. Quotes and aggregate economic deltas are recomputed, not duplicated as independently authoritative numbers. Entries are sorted lexicographically by transaction-ID bytes then numerically by output index; snapshot economics do not use that order as a sequential pricing rule.

The request and mint inner actions are:

```
Request acknowledgment: C(0,[state_ref])
Controller cancellation: C(1,[refund_output])
Expiry refund:           C(2,[refund_output])
Mint initialization:     C(0,[config_output,state_output])
Mint supply update:      C(1,[state_ref])
```

Claim delivery is `C(0,[entries])`, where each `DeliverEntry` is `C(0,[claim_ref,receiver_output])`. Every consumed claim at Q must be covered exactly once, and every claim input must use the same semantic delivery-entry list. That rule is a normative candidate requirement; the local tests exercise allocation primitives, not the full multi-handler claim implementation. The exact recursive schemas are included in `schema.json` and `wire.cddl`.

The Batch reward key must occur among the transaction's explicit required signatories and, for a KeySet policy, in the immutable authorized set. When the sum of request settlement fees is zero, the reward-output option must be None. Otherwise it must select one distinct output to that key's enterprise address, with NoDatum, no reference script, and at least that summed lovelace amount; any excess is an external minimum-output top-up, not an additional request charge. All other assets are excluded from that reward output. The per-request fee is immutable intent; the operator cannot choose a larger amount by changing the reward index.

10.5 Semantic equality and commitment serialization

Plutus Data has integer, bytes, list, map, and constructor forms. Its decoder supports multiple CBOR spellings for some identical Data values. Constructor alternatives 0..6 use tags 121..127 in the preferred representation; 7..127 use 1280..1400; the extended form uses tag 102. Long byte strings use chunks no longer than 64 bytes. [16], [17]

The local preferred writer emits empty lists as definite empty arrays, nonempty lists as indefinite arrays, maps as definite maps in their existing pair order, shortest integer encodings, and the documented byte chunks. A second JavaScript implementation checks these fixtures. **Agreement with the pinned Plutus builtin has not been executed.** This is a release gate, not an inferred pass.

For example, these decode to the same `C(0,[1,2])`:

```
preferred:            d8799f0102ff
definite field array: d879820102
```

Ordinary scripts receive the decoded value, so the reference validator must check constructor shape and semantic constraints, not an alleged original CBOR spelling. Indexers preserve original transaction bytes and IDs; they must not normalize a signed transaction and reuse its original signature/ID.

Terms are committed using the named builtin serialization of validated K and the fixed ASCII domain including its zero terminator. The local encoder provides candidate expected bytes. Do not substitute a generic JSON hash or a generic canonical-CBOR library's defaults. A semantic terms commitment and an actual ledger datum hash are different operations; do not assume reserialization reproduces every original hash spelling.

Protocol-owned records use constructors, not maps with loosely interpreted field names. Opaque recipient Data is not converted to a dictionary or sorted. Its pair ordering is preserved. This bounded candidate rejects duplicate equal map keys in opaque data rather than silently dropping one; supporting a different recipient convention requires an explicit profile. CIP-57 provides schema machinery but does not supply these transaction semantics automatically. [7]

10.6 Operational bounds and decoding gates

Item	Candidate limit
Economic stored quantity	0..2^63-1, plus aggregate physical-value bounds
Output-reference index	0..65,535 and actual-container bounds
Opaque recipient Data	depth <=16; nodes <=256; normalized bytes <=1,024
Opaque integer	signed 256-bit range
State datum	normalized bytes <=256
Terms / config	normalized bytes <=4,096 / 4,608
Settlement-eligible request / claim	normalized bytes <=4,096 / 2,048
Entries and configured settler keys	<=16

These are finite design envelopes selected for testing, **not measured safe transaction capacities**. User datums and whole transaction envelopes require actual execution measurements before any stable maximum is advertised. Structural bounds should be checked before expensive recursive normalization where possible.

Recovery does not require its economic body to pass the settlement byte/type limits. It validates the supported recovery fields. Extremely large or otherwise unconstructible escrow outputs still have no unconditional recovery guarantee; full-value preservation and practical transaction feasibility are separate. A malformed *recovery* envelope or unknown outer version has no implied refund path.

10.7 Machine-readable agreement and the remaining wire gate

`schema.json` fixes constructor types, arities and field order for CTVS-WIRE-2. Additional constraints in this specification and the recorded local schema checker include commitment verification, key ordering/uniqueness, body interpretation, value bounds and opaque-data limits. A type-correct datum is not thereby an authentic or executable transaction. `wire.cddl` is a readable companion, not a successfully executed CDDL-engine validation.

The preferred fixture writer produces candidate expected bytes, while the decoder accepts supported equivalent Data encodings. Arbitrary CBOR types are not valid Plutus Data. Opaque constructor tags are additionally restricted to 0..127; opaque integer range is $-2^{255} \leq x < 2^{255}$; duplicate equal map keys are rejected without lossy dictionary conversion. Recovery's economic body is deliberately not required to satisfy recipient-data or settlement-body limits before dispatch.

The 22 local fixtures and 44 cross-language comparisons are not builtin-certified golden bytes. Compare the exact nested types with the pinned Aiken decoder, compare commitment bytes with `serialiseData`, and obtain independently expected semantic vectors before freezing the encoding. Those E3/E4 results remain unverified. CTVS-2 Section 12 supplies the Request, Recovery, RequestBody and Claim records using these same primitives. [13], [22]

11. Managed accounting and additional fee models

11.1 A separate evidence contract

A vault that moves backing into another protocol is not direct custody. It must advertise a managed profile with an authenticated position set, liability perimeter, valuation method, liquidity estimate, and update authority.

Let G be gross recognized value inside a precisely defined perimeter, D external debt, and Q other payables. Where pending funds, claim reserves, fees, and excluded storage are included in G, a possible reconciliation is:

$$A = G - D - Q - F - P - C - R_{\text{valued}}. \tag{16}$$

This is a perimeter-dependent identity, not an instruction to deduct every symbol twice. If net equity is negative, the managed profile must retain the signed deficit and liabilities, mark insolvency, and halt ordinary conversion-based admission. It must not encode a negative backing value into the nonnegative reference kernel or hide debt by merely reporting zero assets. A separately escrowed claim that was excluded from G is not deducted again. An underlying-denominated lending receivable can be recognized once; its collateral and receipt token must not both be counted as independent backing.

A verified `Price` conversion can value an authenticated position within this perimeter. The Section 6.5 conversion contributes 550,000,000 underlying units to G only when the vault proves custody and the mapping recognizes that position once. It does not establish the whole vault's A or L. Debt, excluded balances and applicable fees remain the managed accounting profile's reconciliation obligations. All components must be valued in one declared asset under a coherent basis; aligning decimal precision is not a conversion between currencies. Receipt conversion evidence is distinct from an absolute whole-vault NAV report, whose update must satisfy Section 11.4.

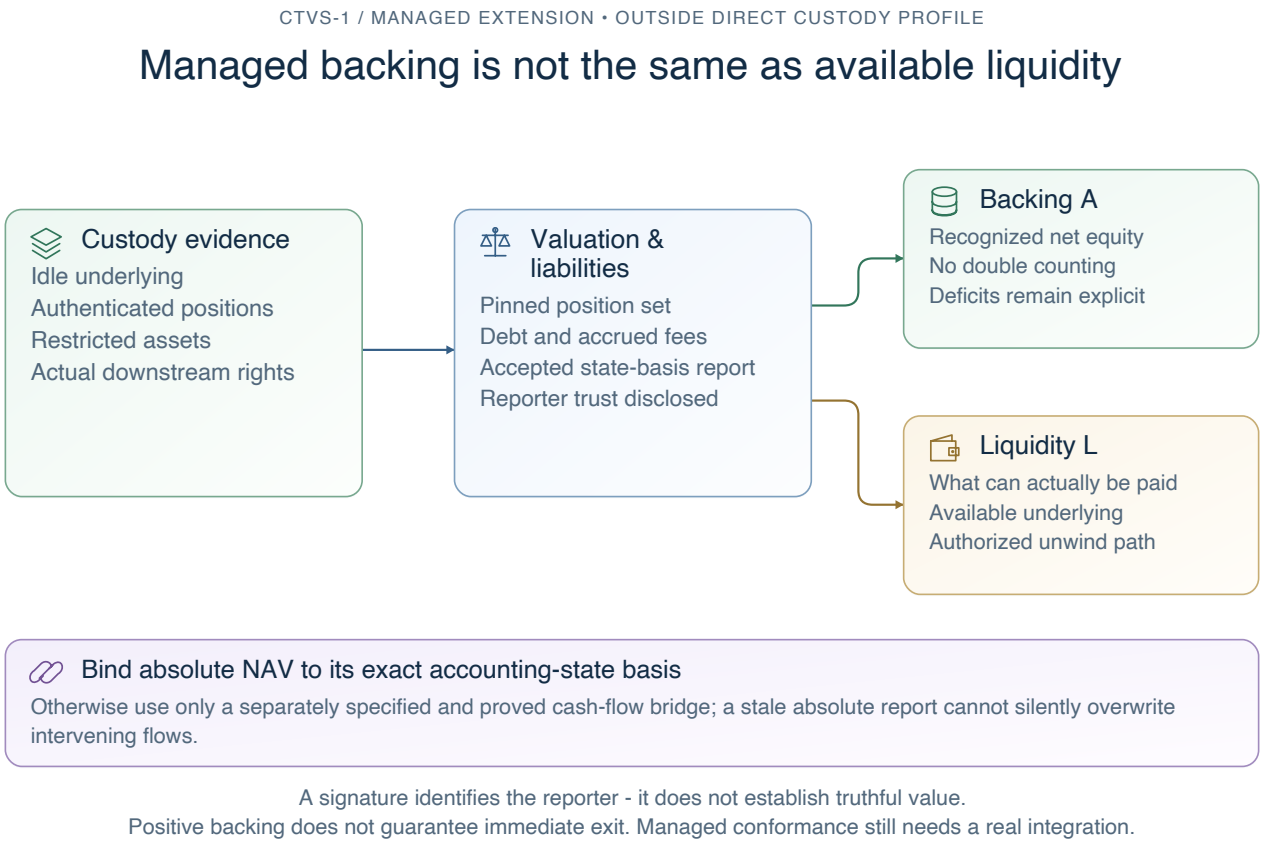


Figure 7: *Managed accounting adds position, liability, valuation, and liquidity evidence. A signed report identifies an attestation; it does not prove fair value or immediate exit liquidity. This is an extension boundary, not Direct Custody Profile behavior.*

11.2 Valuation and loss

A checkpoint identifies position evidence, valuation currency, accepted report epoch, valuation time, expiry, liabilities, and liquidity constraints. A signature proves who attested; a commitment proves which data were committed. Neither proves economic truth. An attested-NAV profile must explicitly disclose the trusted reporter and any independently enforced bounds.

Reference-only old reports can remain unspent. Freshness therefore requires an authenticated accepted-report head or equivalent sequencing rule, plus validity-interval checks. Settlement cannot choose any historically signed report just because its UTxO exists. CTVS-2 describes immutable batch pricing and committed-epoch extensions.

A loss reduces backing without reducing shareholder units unless the published loss model says otherwise. A vault with positive supply and zero recognized backing is not a new bootstrap vault. The baseline managed extension must halt ordinary conversion-based entry until a separately defined recapitalization or loss procedure is authorized. A lack of immediate liquidity is not automatically zero NAV.

11.3 Additional fees: specify the policy, not an incomplete formula

The Direct Custody Profile's fixed entry/exit markup is the only complete fee algorithm fully specified for the reference candidate. A native implementation charging management, performance or share-denominated fees must specify accrual, capital-flow treatment, rate basis, rounding, crystallization order, dilution or asset payment, high-

water-mark/loss treatment where applicable, and replay prevention. Backing, fee liabilities, supply and preview outputs must agree on that same policy.

No illustrative management-fee equation is adopted here. Such a formula without the surrounding timing and ownership rules is not an extension specification. No user flow may be retrospectively charged under new terms it did not authorize. Keep exact tests with the implementation-specific fee definition rather than adding a general fee-module framework.

11.4 Exact state-basis binding and managed acceptance

The common requirement is coherent valuation and complete flow reconciliation, not a prescribed oracle. The following exact-state rule is the conservative absolute-report candidate. A native protocol with provably equivalent state-coupled accounting may map that mechanism instead, but it must demonstrate the same prevention of stale valuations erasing or double-counting user flows.

The conservative absolute-NAV candidate MUST bind a report to the exact accounting-state reference it values, terms version, and accepted report head/sequence. Its update consumes only that basis state. A report for state s0 at 100 must not overwrite successor s1 at 110 after a ten-asset admission with 100. Reject and revalue, or separately specify and verify a cash-flow reconciliation mechanism. An old unspent signed report is not proof of the latest accepted assertion.

The entire transaction interval must fit the accepted valuation window. Exact state binding can stale reports during concurrent user operations, so its operational cost is a measurement obligation. A signature does not establish fair NAV or eliminate reporter trust.

The ten-field reference State has no managed extension slot. Define a distinct exact state/profile for custody/position sets, signed deficits, debt, accepted valuation, liquidity and authorized updates; do not insert these fields into CTVS-WIRE-2 Direct Custody Profile. Before advertising managed conformance, one pinned permissioned protocol integration must demonstrate admission, allocation, position authentication, valuation, liability recognition, liquidity realization, redemption, and failure/recovery. No requirement forces that protocol to open its batcher set. [22], [12]

11.5 One concrete invested-vault mapping before adoption claims

Prioritize one actual single-underlying lending or liquidity-management vault rather than designing every strategy. Its mapping must pin the target version and follow admission, position custody, liabilities and accrued fees, valuation, liquid exit capacity, authorized unwind, redemption and failure/recovery. Distinguish the asset-to-share exchange rate from any external reference currency used to value collateral.

The acceptance trace must use the actual permissioned or permissionless rights of that target, include one loss/illiquidity or stale-accounting negative case, and be reconstructed by an independent client. A positive direct-custody fixture or a parameter named `managed` does not complete this requirement. Until then report managed execution as not evidenced, not as a supported native integration. See requirement REQ-13 in the historical validation evidence.

12. Formal properties and proof sketches

12.1 State and supply induction

Property S1 - unique live state. Assume genesis creates one state token, no later branch can mint or burn it, and every accepted state spend recreates exactly one permitted successor containing it. By induction on accepted transitions, there is one live authenticated state. Unspendability and liveness are separate: uniqueness does not guarantee an operator can or will advance it.

Property S2 - supply conservation. Assume zero genesis share supply, all nonzero share mint/burn requires a valid accounting-state transition, and that transition verifies its supply delta. Induction gives $S = N$ at every accepted state. The state validator must also verify gross economic obligations when the net mint is zero.

These arguments depend on compiled policy/validator enforcement. They are not machine-checked proofs of the unimplemented scripts.

12.2 Value conservation under each action

For deposit, $A' + F' = A + F + n + f = A + F + d$. For redemption, $A' + F' = A + F - g + f = A + F - w$. Fee collection subtracts F from custody and the fee liability together. Reserve top-up affects only reserve ADA. Combined with correct destination allocation, these identities preserve the declared owners of all protected value.

Ledger balance alone does not prove these statements: the validator must enforce the semantic partition and the payment mapping.

12.3 Rounding favors the remaining pool

For a deposit admitting n and issuing $q = \text{floor}(nY/X)$, define $\epsilon = nY - qX \geq 0$. Then:

$$\frac{X+n}{Y+q} - \frac{X}{Y} = \frac{nY - qX}{Y(Y+q)} \geq 0. \tag{17}$$

For redeeming q with gross assets $g = \text{floor}(qX/Y)$, where successor denominators are positive:

$$\frac{X-g}{Y-q} - \frac{X}{Y} = \frac{qX - gY}{Y(Y-q)} \geq 0. \tag{18}$$

Ceiling-priced mint and withdrawal have analogous inequalities. These are statements about adjusted pricing under the specified transitions, not a promise of positive investment returns. Management losses and valuation changes can decrease price.

12.4 Deposit/redeem round trip

At unchanged external valuation, issue $q \leq nY/X$ for admitted assets n. An immediate redemption after that deposit returns gross value at most n, because $qX \leq nY$ implies $q(X+n)/(Y+q) \leq n$. Exit fees can only reduce the net result. Therefore this isolated round trip cannot return more than the original gross deposit under the defined arithmetic and valid states.

The claim excludes strategy P&L, rewards, external subsidies, checkpoint changes, and unrelated portfolio effects. Bounded tests supplement this algebra but do not establish unrelated authorization or ledger properties.

12.5 Role separation and observable-data assumptions

Conditional role-safety argument: if genesis establishes the protected STATE identity, every spend first checks that asset at its own resolved input, and STATE-bearing inputs can select only a valid State action, then a Request constructor cannot route genuine state custody into fixed refund. Reject invalid token quantities and unexpected ID-bearing inputs before dispatch. The local boundary tests exercise selected symbolic counterexamples; compiled dispatch remains an essential premise.

Conditional encoding argument: semantic field validation acts on decoded Data. Alternative accepted CBOR spellings that decode to the same Data must have the same field interpretation. The terms commitment is defined over the named serialization of validated Terms, not the original signed transaction spelling. Agreement of local writers does not prove agreement with the Plutus builtin.

These properties supplement, not replace, state/supply induction and rounding proofs. The economic predicates and byte/link predicates are separate programs; no completed proof connects their composition to actual compiled validator acceptance. [12], [22]

13. Common integration contract and native interpretation

13.1 One verifiable integration boundary

A conforming reader resolves one underlying asset, its share asset, authenticated deployment/terms and their economic interpretation at a coherent chain point. A conforming executor additionally constructs and verifies each claimed operation without depending on an opaque provider for its rules. A fixed API transport, vendor, module framework and arbitrary downloadable builder code are not required.

The Direct Custody Profile resolves its exact template/build, genesis, ID/STATE/SHARE and Config as specified earlier. A native implementation resolves the objects and guards in its reviewed Section 2.5 mapping. Both must support share-to-vault lookup with verified network and implementation identity. Labels, ticker symbols, a registry listing, descriptor metadata or a self-reported conformance badge are discovery hints, not proof of custody or execution rights.

13.2 Required response fields and value states

The following common semantic contract applies regardless of transport. An implementation must publish the fields and meanings of its responses, including the evidence needed to check them. In the pinned Direct Custody Profile source, types.ts defines typed responses and reader.ts reads accepted-chain records. Their `schema` field uses `CTVS-INTEGRATION-0.6` to identify this off-chain response family. That identifier is not an on-chain wire version or a universal JSON schema, and native mappings need not use the reference's field names or serialization.

Response	Minimum required content
Every response	Response-format identifier; record kind; network identity; chain point; vault ID; implementation binding; claimed conformance; evidence references; verification disposition.
Snapshot	Underlying/share IDs; quantities and their units; economic supply and backing; excluded fees; liquidity; status/capabilities; pricing/terms source references.
Quote or estimate	Operation and exact amount; fee-inclusive action result or explicit unknown; user bound; snapshot/pricing basis; economic and construction limits; economic fees, execution charge and protected storage separately; expiry/prerequisites.
Async request/claim	Origin and current lifecycle; economic participation; protected asset/quantity; terms, controller and both destinations; fixed recovery and delivery rights; settlement/claim evidence and fee/reserve allocation.
Build effects	Exact operation and approved bound; inputs/reference inputs, mint/burn, expected outputs/datums, change, fee/collateral exposure, required authority, validity interval and evidence-verifiable effect summary.

Illustrative response interpretation. In Section 9.2, a quote against State `s0#0` for a 101,000 UNIT deposit reports a 1,000 UNIT economic fee and 100,000 SHARE output under that State, with the user bound of at least 99,900 SHARE. Before transaction acceptance, this is a quote tied to its chain point and State reference, not a realized deposit. After a valid accepted transaction, the 100,000 SHARE and 100,000 UNIT admitted backing are realized effects with transaction and output references. The example supplies no authentic chain evidence by itself; a reader must obtain and verify that evidence separately.

The linked response types and transport.ts serializer are one concrete Direct Custody Profile implementation of the common semantic fields above. They do not certify a generic external JSON schema or replace the requirement to disclose unknown values and verification limits.

Unsigned quantities are decimal strings. Assets use `ada` or explicit policy-ID/asset-name bytes; display symbols and decimals do not change arithmetic. A quantity result is `known` with a value, or `unknown`, `unsupported`, `stale` or `not_applicable` without a fabricated value. Known zero is not missing data. Native signed deficits, if relevant, must be separately represented by the managed mapping rather than cast into unsigned backing.

For a `Price` integration, disclose the approved provider script identity/version, supported assertions, evidence schema, native source identities, scale interpretation and pricing basis through the implementation binding. Display decimals and their metadata source are separate presentation information. The provider's code-reference location is an availability hint; clients must verify its script identity rather than trust the location.

Capabilities are operation-specific. Read availability does not imply construction or execution. Each direction reports supported operations, delivery form where async, applicable pricing rule, cancellation rights and authority requirements. An unknown required native rule disables execution; a provider may still return an explicitly partial or unsupported read view.

13.3 Snapshot, quote and limit invariants

All references must belong to the same verified ledger history and chain point. Identity/terms sources cannot be substituted from a newer incompatible configuration. A direct quote's pricing basis identifies the consumed state evidence or the native equivalent; an async request estimate never silently fixes a future rate. `exact_at_snapshot` means equality under that snapshot's declared calculation, not a reservation or guaranteed inclusion.

Report an economic minimum and maximum separately from a construction limit. A resource-feasible maximum cannot be inferred from Section 6.4's arithmetic search. `paused`, `unavailable_liquidity`, `unsupported`, `unknown` and `requires_fresh_build` are different explanations. Read-only estimates must not report a transaction as ready. Every constructed action still commits the appropriate min-output/max-input bound and intended destinations.

Fees are explicitly classified: protocol-economic fees in assets or shares under the declared algorithm, execution charge, refundable/carry storage, network fee estimate and any externally funded top-up. Their asset units, recipients and point of accrual must reconcile with the action. A quote MUST NOT add the same fee to a liability and deduct it again in its net payout.

13.4 Realized results, verification and rollback

A quote, an authorized request and an accepted result are separate records. Realized events identify actual transaction and source references, operation, gross and net effects, fees, participating-share delta, assigned outputs and terms/pricing evidence. Internal route-step attribution is optional. Missing per-step data must remain unknown and must not be replaced with predicted values.

Before signing, verify user inputs, amounts, bound, receiver and datum, change, state/native identity, mint/burn, fee/storage partitions, interval and required witnesses, including collateral exposure. A hosted builder is not a trusted signer. Preserve original signed bytes/IDs; do not normalize transaction CBOR and reuse its signature.

Rollback reverses both status and quantities. An orphaned settlement loses its realized output/claim attribution; a request is pending only if it exists on the replacement chain. Reversing delivery alone restores the unspent claim. Pending deposits, locked participating redemption shares, issued shares awaiting delivery and fixed asset claims must remain separate so wallets cannot double-count them. These are reconstruction requirements, not a mandatory event-DAG format.

13.5 Implementation binding and the integration acceptance test

`implementation` identifies either the Direct Custody Profile's exact reference binding or a native implementation's mapping ID, mapping version, mapping commitment and source references. Every independently decoded field must trace to that binding. The hash algorithm, domain and exact bytes covered by a mapping commitment must be documented by the mapping; merely hashing an arbitrary provider response is not authentication. No universal mapping registry or code runtime is introduced.

An independent builder must derive the same permitted effect from the public mapping, not call the first builder's opaque construction service. A separately implemented decoder/indexer must agree on native bytes, economic field meanings, each supported operation and real rollback. A well-shaped response is not that acceptance test. The linked Direct Custody Profile reader and its local tests are implementation evidence, not independent interoperability or a declaration that a native protocol conforms. [20], [21]

A `Price` construction recipe must locate current authenticated native evidence and consumer inputs, obtain the approved program or code-reference output, calculate and encode assertions, include the evidence reference inputs and zero withdrawal, construct protected outputs, then balance and evaluate with actual fees, collateral and witnesses. Chain-data access is required; operating a dedicated indexer or using a particular WASM runtime is not. A missing or spent source requires fresh discovery and rebuilding, not silent reuse of a cached claim. Section 6.7 and the isolated example supply a shared codec and signed local construction. The existing reader serves the Direct Custody Profile; production native discovery and independent-provider interoperability remain separate work.

14. Resource limits, operational behavior, and risks

Minimum ADA depends on the actual serialized output and protocol parameters. CIP-55 gives the Babbage-era expression $(160 + \text{byteLength}(\text{serializedOutput}))$ coinsPerUTxOByte. Implementations must use the active ledger-era calculation and parameter snapshot, not the numerical reserves in this paper. [6]

Measure datum sizes, worst-case integer serialization, asset names, inline recipient datums, reference inputs, witnesses, execution units, and transaction size. Bound every list and arbitrary datum field. Reference scripts improve witness reuse but require availability monitoring; their output references are hints that may become stale.

Risk	Required interpretation or control
State contention	Rebuild against authenticated successor; do not weaken signed bounds.
Donation/spoofed UTxO	Count only authenticated state and permitted accounting transitions.
Dust/residual rounding	Reject zero-output execution; disclose immutable residual policy.
False managed NAV	Disclose reporter trust and independent checks; do not call it direct custody.
Output reuse	Complete allocation plus the enforced composition profile.
Authority failure	Distinguish pause rights, asset movement, valuation, and execution roles.
Provider inconsistency	One-chain-point snapshots; independent decoding and rollback.
Protocol changes	Recompute budgets and reserves; do not change economic terms silently.

No throughput, cost, yield, or unconditional exit guarantee is made. The reference profile favors a small validation surface over unrestricted atomic composition. That tradeoff must be visible to integrators.

14.1 Reference bounds are not universal or measured capacities

The representative candidate fixture datums are Config 333 bytes, State 92, Request 310 and Claim 311. These are neither worst-case encodings nor complete outputs, and they do not establish minimum ADA or execution cost. Use the active ledger's serialized-output calculation. A top-up can itself increase encoded size; rebalance and recompute until the complete output/transaction meets its actual constraints. Config remains referenced rather than rewritten; a valid state reserve-top-up may recreate state when necessary. [22], [6]

Measure all permitted action paths, reference witnesses, worst-case credential/data shapes, quantity growth, batch lists, required keys, collateral and collateral return. Prefer central batch economics plus explicit per-request linkage, but benchmark the actual handlers: repeated full-list scans may multiply cost. Structural cap 16 is not promised throughput.

The Direct Custody Profile restrictions are deliberate interoperability limits: no foreign script-input composition, no pointer/bootstrap destination, no datum-hash destination, no protected attached reference script, and no script controller. A later profile can revise them only with explicit semantics, versioning and evidence. No capability or managed support follows from naming an extension. [12]

15. Validation and release requirements

15.1 Acceptance matrix

This is the concrete reference-profile acceptance matrix. A native implementation must prove the corresponding common obligations through its Section 2 mapping, not reproduce the reference factory or every reference-local mechanism. The script-controlled and managed integration targets have additional requirements in Sections 7.5 and 11.5. Evidence for one construction cannot be transferred by analogy to another.

ID	Requirement	Required evidence
S-01	Authenticated genesis and unique identity	Compiled genesis vectors; duplicate/extra mint rejection
S-02	Immutable terms and correct state succession	Valid/invalid contexts for every state branch
S-03	Economic and native supply agree	Mint, burn, alternate-burn, and zero-net cases
S-04	Closed ADA/native-token value partitions	Conservation tests with fees, reserves, and top-ups
S-05	Exact integer quotes and bounds	Independent mathematical vectors and edge states
S-06	Complete payment attribution	Same-destination, reused-index, and cross-vault rejection
S-07	Correct authority execution	Actual key witnesses; capability proofs for any advertised extension
S-08	Explicit residual and insolvency behavior	Bootstrap/residual cycles and loss-profile tests
S-09	Native schema interoperability	Golden CBOR; at least two independent decoders

ID	Requirement	Required evidence
S-10	Reproducible transaction construction	Independent builders and ledger-valid transactions
S-11	Bounded resource usage	Measured bytes, CPU/memory, fees, and reserves
S-12	Reliable infrastructure views	State lineage, coherent snapshots, and rollback replay

An implementation advertising the optional `Price` extension additionally requires mixed-decimal and differently scaled source vectors; fractional and exact-output rounding; incorrect or unbound scale rejection; signed subject/scale/freshness rejection when signatures are supported; unsupported and duplicate assertion rejection; and provider/consumer selection and custody tests. End-to-end acceptance must supply actual compiled provider code, invoke both purposes in a signed ledger-valid transaction, exercise stale/replaced evidence, and measure resources. Local synthetic handler tests alone do not establish that acceptance. These requirements do not assert new tests have been executed for this publication.

15.2 Recorded evidence and publication checks

Both papers refer to one shared evidence record in historical validation evidence; the historical results are not new runs and are not additive across the papers. They cover bounded economic/semantic checks and separate candidate-codec/link checks, not a complete compiled validator. Tests and models share an author; two languages do not establish independent review. [22]

The historical publication check covered candidate-wire preservation, arithmetic examples, illustrative response fixtures, source references, and layout. Its saved results and preflight reports are preserved with the prior edition under publication archive. The historical suites do not establish that their illustrative response schema matches the later reference implementation. The linked implementation has separate compiled-validator and signed local transaction evidence in [21]. Node-backed acceptance, independent interoperability, external review, and deployment remain unverified.

The historical requirement register retains its recorded BLOCKED dispositions; this paper does not rewrite those dated results. The later Direct Custody Profile reference supplies compiled validators and signed local examples for direct operations, requests, settlement, recovery, and Claim delivery [21]. It does not establish node-backed lifecycle acceptance, full resource capacity, independent builder or indexer agreement, managed integration, or production security. Treat each implementation claim according to its own pinned source and evidence, not the older register alone.

15.3 Required model-to-validator refinement

Let $\alpha(T)$ project a real transaction and its resolved ledger context into the specified semantics. The soundness target is:

$$\begin{aligned} &\text{LedgerPrerequisites}(T) \\ &\quad \wedge \text{CompiledValidatorsAccept}(T) \\ &\qquad \Longrightarrow \text{CTVSRequirements}(\alpha(T)). \end{aligned} \tag{19}$$

Define every projection: own script purpose, ordinary versus reference inputs, credential nesting, actual datum variants, required signatories, mint, auxiliary era fields, time bounds, source references and output indices. Decoding bytes and passing amounts into the semantic model by hand is not that refinement proof.

Constructive success is also required: each advertised supported operation has a valid, resource-feasible, ledger-accepted transaction with the intended effect. Reject-everything scripts are not conforming. Negative cases must fail for the intended semantic reason; phase-1-invalid witnesses must not mask missing phase-2 payout checks.

15.4 Compiled end-to-end acceptance trace

The trace below validates the specified **reference construction**. A native mapping supplies its own equally explicit deployment, operation and recovery trace under its actual scripts and custody assumptions; it is not required to deploy F_lock or Q_claim.

```
authenticated F_lock / Q_claim / per-vault T build and genesis
 -> direct deposit with exact State/Config/Terms
 -> independent deposit and redemption Request(Recovery,Data)
 -> purpose-bound permissioned Batch, including zero-net mint
 -> funded Claim creation and fixed delivery to key/script destinations
 -> cancellation/expiry with invalid economics and batcher unavailable
 -> fee collection and state-reserve top-up
 -> independent construction, authentic lineage and rollback replay
```

Run the same trace against malformed encodings, equivalent Data encodings, wrong-role datums, substituted config, wrong-branch acknowledgments, omitted/duplicate requests, output reuse, balanced theft, stale references and forbidden auxiliary actions. Confirm the pinned builtin's commitment bytes and actual ledger datum/hash behavior. Record full transaction bytes, outputs, witnesses, fees, collateral/return, minimum ADA and CPU/memory under a named protocol-parameter snapshot.

The baseline recorded in the repository is Aiken v1.1.22, standard library v3.1.0 and Plutus V3. This identifies the intended baseline, not the latest tooling or a successful compilation. Pin the ledger implementation and network parameters as well. [22], [14]

15.5 Profile-specific and independent-integration gates

A managed-profile claim additionally requires a concrete permissioned position lifecycle with exact custody, liabilities, accepted NAV basis, liquidity realization and real recovery permissions. A script-controller claim requires actual guard/purpose/domain and successor tests. Neither follows from direct-custody or key-only evidence.

A second builder must construct from authenticated public data without calling the first builder's construction implementation. Separate decoder expectations and a separate indexer must agree on transactions, exact fields, origin attribution and rollback. Two wrappers around one library or two codecs by the same author are not independent external validation.

The shared evidence register records local evidence without clearing any compiled-validator or ledger gate. The reference wire remains a candidate. Publishing this document does not update GitHub, deploy a contract, or certify production security.

16. References

ERC-4626, ERC-7540 and CIP-57 were rechecked on 11 September 2026 for their interface context; no fresh contract or deployed-protocol audit was performed. New scope, mapping and delivery requirements are CTVS proposals, not claims of SundaeSwap conformance. External protocol documentation describes published designs and is not a certification of deployed instances. The equations, proposed profiles, worked examples, and proof sketches in this paper are the proposed CTVS design unless explicitly attributed otherwise.

[1] Steer Protocol. Proposed wire format and accounting kernel, `lib/ctvs/wire.ak` and `lib/ctvs/math.ak` in the reviewed repository snapshot.

[2] ERC-4626: Tokenized Vaults. https://eips.ethereum.org/EIPS/eip-4626

[3] ERC-7540: Asynchronous ERC-4626 Tokenized Vaults. https://eips.ethereum.org/EIPS/eip-7540

[4] Cardano CIP-31: Reference inputs. https://cips.cardano.org/cip/CIP-0031

[5] Cardano CIP-32: Inline datums. https://cips.cardano.org/cip/CIP-0032

[6] Cardano CIP-55: Protocol Parameters (Babbage Era). https://cips.cardano.org/cip/CIP-0055

[7] Cardano CIP-57: Plutus Contract Blueprint. https://cips.cardano.org/cip/CIP-0057

[8] Cardano CIP-40: Collateral Output. https://cips.cardano.org/cip/CIP-0040

[9] SundaeSwap. Published contract architecture. https://github.com/SundaeSwap-finance/sundae-contracts

[10] Minswap. AMM V2 specification. https://github.com/minswap/minswap-dex-v2/blob/main/amm-v2-docs/amm-v2-specs.md

[11] OpenZeppelin. ERC-4626 documentation. https://docs.openzeppelin.com/contracts/5.x/erc4626

[12] CTVS-WIRE-2 reference encoding. Exact layouts and transaction obligations are specified in Section 10. Supporting artifacts are listed in [13].

[13] CTVS-WIRE-2 candidate artifacts: schema, CDDL, golden bytes, and Aiken types. These candidate type declarations are uncompiled; the candidate bytes have not been certified against the Plutus builtin.

[14] Aiken standard library v3.1.0, cardano/transaction.ak. Recorded blob `12f8ec80fdfe6c9976d63f3bb701f4fd525d3ec3`. https://github.com/aiken-lang/stdlib/blob/v3.1.0/lib/cardano/transaction.ak

[15] Aiken standard library v3.1.0, cardano/address.ak. Recorded blob `0167b90fbbd4ad063db2725c44023f8d0bab42d6`. https://github.com/aiken-lang/stdlib/blob/v3.1.0/lib/cardano/address.ak

[16] Cardano CIP-42, serialiseData. https://cips.cardano.org/cip/CIP-0042

[17] IntersectMBO Plutus, PlutusCore/Data.hs. Recorded source blob: `a994354fb3c87a55ed6d1b4b6b544c7cd94c1c60`. https://github.com/IntersectMBO/plutus/blob/master/plutus-core/plutus-core/src/PlutusCore/Data.hs

[18] Aiken, Common Design Patterns. https://aiken-lang.org/fundamentals/common-design-patterns

[19] Steer Protocol. CTVS-1 and CTVS-2 reference workspace, pinned at repository commit `b52962987d3d078d3da7ba05ca973fe2926b101e`. Source and local tests, not independent conformance certification.

[20] Steer Protocol. Integration response types, accepted-chain reader, transport serialization, and reader tests at the pinned commit. The response shape is implementation-specific.

[21] Steer Protocol. Reference validation record at the pinned commit. Dated local results and remaining acceptance work.

[22] CTVS historical validation evidence. Summarizes semantic and economic checks, candidate-codec checks, and implementation assurance requirements with their recorded limitations. Historical requirement dispositions remain BLOCKED; current implementation results are separately documented in [21].

Appendix A. Scope and delivery checklist

Decision from the scope review	Specification
Preserve exact UTxO/datum hardening	Direct Custody Profile reference construction and all candidate-wire files retained
Stop treating one construction as universal	Common guarantees separated; native-format conformance now explicit
Avoid new transcript/module/permission frameworks	None required by this specification
Keep economic supply and user-bound mathematics	Retained; native inventory/fee rules require precise mappings
Support script-controlled protocol users	Concrete acceptance target, not a fictitious key-only success claim
Avoid mandatory intermediate claim for every native flow	Bounded direct-at-settlement semantic variant added outside the Direct Custody Profile
Consider budget simplification	Algebra checked; existing fields preserved, no silent wire change
Trim unfinished features and duplicated evidence	Epoch/partial-fill workflow algorithms deferred; evidence consolidated once
Make invested-vault support substantive	One pinned managed integration is an explicit unresolved acceptance milestone

The publication source bundle includes both Markdown papers, figure sources, unchanged candidate-wire artifacts, the shared historical evidence record, and PDF build tooling. `python tools/build_whitepapers.py` typesets the papers. The archived publication checker records historical checks; current reference implementation code and dated local validation are linked in [19] and [21]. Final publication QA must inspect rendered equations, tables, headings, cross-references, and links.

Appendix B. Candidate Price encoding

Exact for this optional candidate, separate from CTVS-WIRE-2. The public type is `Price`; its encoded version is 1. It is a withdrawal redeemer, not a Direct Custody Profile State or action. Section 6.6 supplies the economic and invocation requirements. Adopting this encoding does not alone establish native execution conformance or a reviewed composition profile.

Use Section 10.1's `C(tag,[fields])`, `Asset`, `OutRef` and `Q` notation. A native asset retains its 28-byte policy ID and 0..32-byte name. `market_id` uses the Asset encoding as an implementation-defined subject identifier; this syntax does not require a singleton NFT or establish source authenticity. Provider and consumer configuration must authenticate and match that subject.

```
Price = C(0,[
    1,                          -- Price version, not wire version 2
    market_id: Asset,
    share_asset: Asset,
    underlying_asset: Asset,
    List<PriceAssertion>,
    evidence: BoundedData
])

PriceAssertion =
    C(0,[shares: Q, assets: Q])         -- ConvertToAssets
  | C(1,[assets: Q, shares: Q])         -- ConvertToShares
  | C(2,[assets_in: Q, shares_out: Q])   -- PreviewDeposit
  | C(3,[shares_out: Q, assets_in: Q])   -- PreviewMint
  | C(4,[assets_out: Q, shares_in: Q])   -- PreviewWithdraw
  | C(5,[shares_in: Q, assets_out: Q])   -- PreviewRedeem
```

The outer constructor has exactly six fields. Every assertion has exactly two fields, with its fixed input first and claimed result second. Unknown versions, tags, wrong types, malformed asset identities and extra or missing fields fail. Quantities are integers in 0..2^63-1 in their respective ledger units; this includes valid zero-valued conversion views, not permission for zero-output execution.

The assertion list is nonempty and has at most 16 entries. Within an envelope, `(operation tag, fixed input)` identifies a query. Duplicate queries fail even when their results match. Providers verify every assertion, rejecting unsupported methods. Consumers select exactly one matching query; list order must not select the assertion.

Evidence is provider-specific Plutus Data. The candidate bound is at most 1,024 bytes under the pinned decoder's normalized serialization, 256 nodes counting containers and map keys/values, and depth 16 with the root at depth zero. Constructor tags are 0..127; byte strings are at most 1,024 bytes; integers are in $[-2^{255}, 2^{255}-1]$; maps reject structurally duplicate keys. These are the shared decoder's resource bounds, not universal ledger Data limits or proof that a maximum-size invocation fits a transaction. The original accepted CBOR spelling need not have the normalized size. Providers must publish their exact evidence schemas within the advertised bounds and measure actual verification costs.

For the hypothetical LendingA read-only example, one evidence schema is:

```
OnChain = C(0,[market_state_ref: OutRef])
OutRef  = C(0,[B[32], output_index])
```

The output index is in 0..65,535. The reference must identify exactly one included reference input; it is not an ordinal selecting the second or third entry in the reference-input list. The adapter additionally verifies the configured state address and market NFT, then decodes its native backing and supply. This example schema is not imposed on other providers. A signed provider may define different evidence, including a report, signature and validity/state-basis data under its published authenticated schema.

There is no common signature field, rate-decimals field, provider hash or liquidity reservation in `Price`. The approved provider identity belongs to the consumer's authenticated configuration; signature and rate-scale interpretation belong to the provider's evidence/configuration contract. Liquidity and execution capabilities remain separately disclosed. No arbitrary builder-supplied decimal exponent may override those meanings.

The Aiken type and decoder, TypeScript codec, interface notes and isolated example are local candidate sources. Section 6.7 provides the complete transaction and decoding walkthrough. The example adds a shared byte vector, native-state verification, custody checks and signed local traces with measured execution. It supplies no signature verifier or deployed native integration. The isolated positive node trace does not establish independent construction or production deployment. Those and provider-specific requirements remain obligations under Section 15.